

Inside the Falcon How CrowdStrike Catches You

 0xdbgman.github.io/posts/inside-the-falcon-how-crowdstrike-catches-you

July 31, 2026

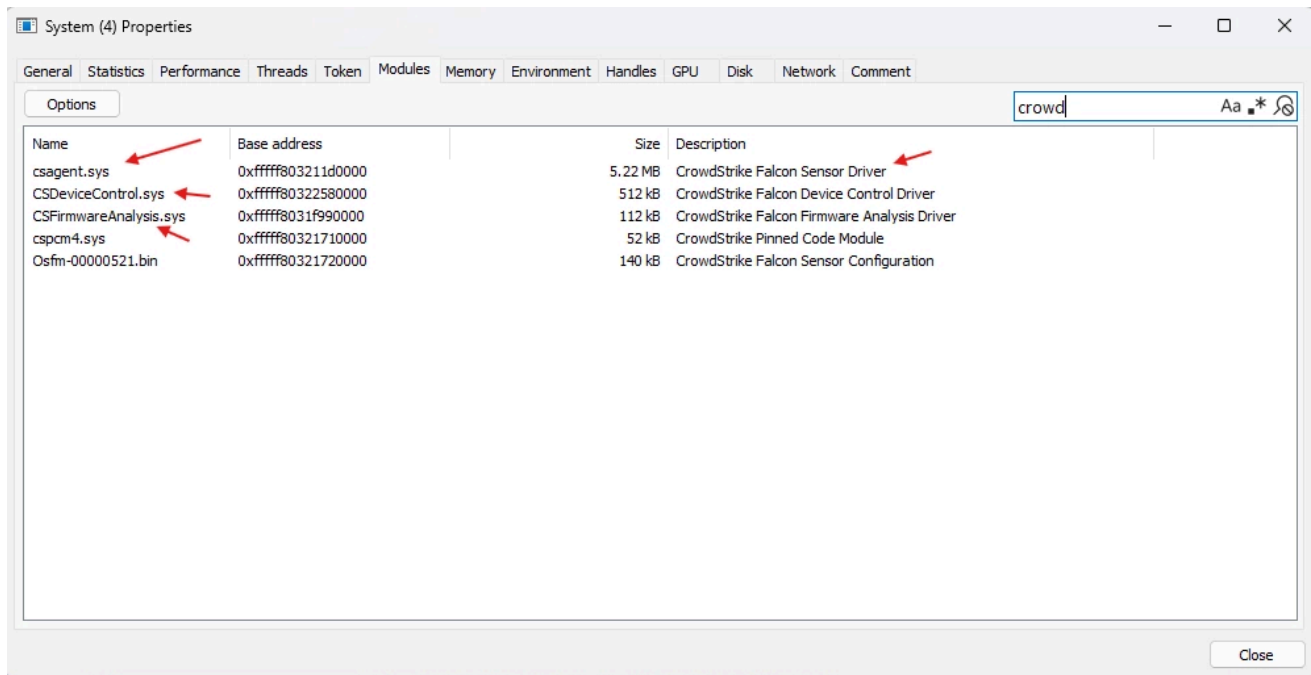


A full reverse-engineering teardown of the CrowdStrike Falcon sensor: the six kernel callback sources, the WFP network engine, the file-system minifilter, the cspcm4 broker, the user-mode service (DNS / AMSI / browser / Identity Protection), and the cloud-loaded detection engine — plus every structural blind spot. How Falcon sees you, and where the seams are.

Section 1 The Components

Kernel-mode

lets see whats Driver There used



user-mode

they used

- CSFalconService.exe
- CSFalconContainer.exe

Name	PID	CPU	I/O total r...	Private bytes	User name	Description	Protection
CSFalconService.exe	3472	0.07	12 B/s	78.8 MB	NT AUTHORITY\SYSTEM	CrowdStrike Falcon Sensor Service	Light (Antimal...
CSFalconContainer.exe	4972			8.51 MB	NT AUTHORITY\SYSTEM	CrowdStrike Falcon Sensor Servlet Container	Light (Antimal...

csagent.sys Reverse

import Dll From csagent.sys

Disasm	General	Strings	DOS Hdr	Rich Hdr	File Hdr	Optional Hdr	Section Hdrs	Imports	Resources	Exception	Security	BaseReloc	Des
✚ + 📄 🔍													
Offset	Name	Func. Count	Bound?	OriginalFirstThun	TimeDateStamp	Forwarder	NameRVA	FirstThunk					
50A78C	ntoskrnl.exe	477	FALSE	5179C8	0	0	518908	29F748					
50A7A0	HAL.dll	2	FALSE	5175A0	0	0	51B3A8	29F320					
50A7B4	FLTMGR.SYS	82	FALSE	517308	0	0	51BC4E	29F088					
50A7C8	NETIO.SYS	15	FALSE	517620	0	0	51BD88	29F3A0					
50A7DC	NDIS.SYS	12	FALSE	5175B8	0	0	51BE8A	29F338					
50A7F0	fwpsclnt.sys	53	FALSE	5176F8	0	0	51C3FA	29F478					
50A804	ksecdd.sys	28	FALSE	5178A8	0	0	51C5E8	29F628					
50A818	ci.dll	3	FALSE	5176A0	0	0	51C634	29F420					
50A82C	msrpc.sys	6	FALSE	517990	0	0	51C6B2	29F710					
50A840	cspcm4.sys	6	FALSE	5176C0	0	0	51C6BC	29F440					
50A854	CLFS.SYS	16	FALSE	517280	0	0	51CA80	29F000					

we can see here 5 important sys

1- FLTMGR.SYS ⇒ its mean Registered minifilter driver to can see all operation file system like Create or Delete ,Ex :**kernel-mode APIs** FltRegisterFilter, FltStartFiltering, FltGetFileNameInformation

29F128	FitGetVolumeName	-	51B800	51B800	-	97
29F130	FitGetVolumeProperties	-	51B814	51B814	-	98
29F138	FitGetVolumeGuidName	-	51B82E	51B82E	-	94
29F140	FitQueryVolumeInformation	-	51B846	51B846	-	CB
29F148	FitRegisterFilter	←	51B434	51B434	-	D3
29F150	FitGetRoutineAddress	-	51B448	51B448	-	86
29F158	FitObjectReference	-	51B460	51B460	-	AF
29F160	FitStartFiltering	←	51B476	51B476	-	FF
29F168	FitReleaseFileNameInformation	-	51B48A	51B48A	-	D9
29F170	FitGetInstanceContext	-	51B4AA	51B4AA	-	76
29F178	FitSupportsStreamContexts	-	51B4C2	51B4C2	-	102
29F180	FitAllocateContext	-	51B4DE	51B4DE	-	8
29F188	FitAllocateGenericWorkItem	-	51B4F4	51B4F4	-	11

2- NETIO.SYS ⇒ used for lower-level network stack to handle the network packets and give him visibility on network traffic

3- fwppkclnt.sys ⇒ is the WFP kernel client 53 imports means csagent.sys has a full network filtering engine that can inspect, block, and modify connections across all layers (transport, network, ALE), giving it firewall-like capabilities to monitor DNS queries and kill C2 traffic.

4- CLFS.SYS ⇒ its a transactional logging; buffers telemetry locally in crash-safe logs and flushes to cloud when connected.

5- cspcm4.sys ⇒ CrowdStrike's own Pinned Code Module; 6 imports used for internal anti-tamper and sensor code integrity protection.

Reverse main csagent.sys

Lets see For Example PsSetCreateProcessNotifyRoutineEx

Address	Ordinal	Name	Library
000000014029F890		PsSetCreateProcessNotifyRoutineEx	ntoskrnl

the process-notify import we pivot on: PsSetCreateProcessNotifyRoutineEx.

Cross Ref

The screenshot shows the IDA Pro interface with the assembly view of the function `PsSetCreateProcessNotifyRoutineEx`. A cross-reference window is open, displaying a call from `sub_1400C370C+7A` to `cs:PsSetCreateProcessNotifyRoutineEx`. The assembly code in the background includes various system calls and conditional jumps, such as `NTSTATUS (__stdcall *IoFreeWorkItem)(PIO_WORKITEM IoWorkItem)` and `NTSTATUS (__stdcall *CmUnRegisterCallback)(LARGE_INTEGER Cookie)`.

cross-references to PsSetCreateProcessNotifyRoutineEx where the callback gets registered.

if not found will do a fallback on PsSetCreateProcessNotifyRoutineEx

```

8  if ( a2 && !*a1 )
9  return 0;
10 SystemRoutineAddress = (__int64 (__fastcall *) (__QWORD, void (__stdcall *) (PEPROCESS, HANDLE, PPS_CREATE_NOTIFY_INFO), __int64)) qword_1402F87E8;
11 if ( qword_1402F87E8
12 || (RtlInitUnicodeString(&DestinationString, L"PsSetCreateProcessNotifyRoutineEx2"),
13 SystemRoutineAddress = (__int64 (__fastcall *) (__QWORD, void (__stdcall *) (PEPROCESS, HANDLE, PPS_CREATE_NOTIFY_INFO), __int64)) MmGetSystemRoutineAddress(&DestinationString),
14 (qword_1402F87E8 = (__int64) SystemRoutineAddress) != 0 ) )
15 {
16 LOBYTE(a3) = a2;
17 ProcessNotifyRoutine = SystemRoutineAddress(0, sub_1400C3310, a3);
18 }
19 else
20 {
21 ProcessNotifyRoutine = PsSetCreateProcessNotifyRoutineEx(sub_1400C3310, p2);
22 }
23 v7 = ProcessNotifyRoutine;
24 if ( ProcessNotifyRoutine >= 0 )
25 {
26 *a1 = a2 ^ 1;
27 return 0;
28 }
29 __mm_lfence();
30 if ( Process != (PEPROCESS) & Process && *((_DWORD *) Process + 11) & 2 != 0 && *((_BYTE *) Process + 41) >= 2u )
31 sub_140088F8((__QWORD *) Process + 3), 19, &unk_1402C5468, qword_1402F87E8 != 0 ? 2 : 0, ProcessNotifyRoutine);
32 return v7;
33 }

```

the driver resolves *PsSetCreateProcessNotifyRoutineEx2* dynamically via *MmGetSystemRoutineAddress*, falling back to the *Ex* variant.

anyway the Callback Function register in `sub_1400C3310` ⇒ `CS_ProcessNotifyCallback`

```

void __fastcall sub_1400C3310(PEPROCESS Process, HANDLE ProcessId, PPS_CREATE_NOTIFY_INFO CreateInfo)
{
sub_1400C3320(Process, Process, ProcessId, CreateInfo);
}

```

the registered callback `sub_1400C3310` (`CS_ProcessNotifyCallback`) a thin wrapper into the real handler.

good we now got some Spicy function last first analysis all in `sub_1400C3320` than Deep Dive in our Function.

Process Creation

```

27 if ( a4 )
28 {
29 result = sub_1400C39A8(v4, 0, (__int64)&v17);
30 if ( (int)result < 0 )
31 goto LABEL_9;
32 v5 = (LARGE_INTEGER *)v17;
33 }

```

process-creation branch: non-NULL *CreateInfo* (*a4*) → `sub_1400C39A8` allocates the internal tracking record.

a4 is a *CreateInfo* if not Null That's Mean process creation

`sub_1400C39A8` do a allocate for internal process tracking record and Fill in the data for the new process and Return to *V17*

Process Termination

`sub_1400C3BEC` ⇒ `CS_GetExistingProcessRecord`

```

34 else
35 {
36 result = sub_1400C3BEC(v4, a2, (__int64)&v17);
37 if ( (int)result < 0 )
38 goto LABEL_9;
39 v5 = (LARGE_INTEGER *)v17;
40 v11 = *(struct _KPROCESS **) (v17 + 56);
41 *(_DWORD *) (v17 + 240) |= 0x10u;
42 v5[37].LowPart = PsGetProcessExitStatus(v11);
43 v5[36] = PsGetProcessExitTime();
44 }

```

termination branch: `CS_GetExistingProcessRecord` marks the record terminated and records exit status/time.

- `sub_1400C3BEC` retrieves the existing internal tracking record for the terminating process.
- extracts the `KPROCESS` pointer from record offset 56.
- sets flag `0x10` at offset 240 marks the record as “terminated.”
- `PsGetProcessExitStatus` stores the process exit code into the record.
- `PsGetProcessExitTime` stores the exact termination timestamp.

Process Blocking

`sub_1400C3DA8` ⇒ `CS_ReleaseProcessRecord`

`if (v18 < 0) → *(a4 + 64) = v18` this is the critical line. Offset 64 in `CreateInfo` is the **CreationStatus** field. If the detection engine returns a negative `NTSTATUS` in `v18`, it's written here, which **blocks the process from being created** this is CrowdStrike's process-kill mechanism.

```
    if ( a4 )
    {
LABEL_21:
        if ( v18 < 0 )
            *(_DWORD*)(a4 + 64) = v18;
LABEL_27:
        _mm_lfence();
        return sub_1400C3DA8(v4, v5);
    }
```

the kill mechanism: a negative `NTSTATUS` written to `CreateInfo+64` (`CreationStatus`) blocks the process from ever starting.

Event Dispatch

```
47  if ( v6 )
48      _InterlockedDecrement((volatile signed __int32 *)v4 + 55);
49  if ( (int)result >= 0 )
50  {
51      v17 = sub_1403787F4();
52      if ( v17 )
53      {
54          _mm_lfence();
55          LOBYTE(v12) = 2 - (a4 != 0);
56          sub_140377450(v5, v12, a4, &v18);
57          sub_140378824(v13, &v17);
58          if ( a4 )
59          {
60 LABEL_21:
61              if ( v18 < 0 )
62                  *(_DWORD*)(a4 + 64) = v18;
63 LABEL_27:
64              _mm_lfence();
65              return sub_1400C3DA8(v4, v5);
        }
```

event dispatch: acquire context, compute event-type ID (1=create, 2=terminate), call the detection/dispatch path.

- `sub_1403787F4` acquires event processing context; non-null return means the detection engine is ready.
- `2 - (a4 != 0)` computes event type ID: 1 = creation, 2 = termination.
- `sub_140377450` main detection/event dispatch; takes process record, event type, and `CreateInfo`, writes verdict to `v18`.
- `sub_140378824` likely telemetry submission or event completion callback.

Deep Dive

Now lets Jump into theses 2 function

1- sub_1400C39A8 (CS_AllocProcessRecord) ⇒ what it's gathering about the new process.

the main function to get info about process it sub_1400C42F4 we found it into sub_1400C39A8 (CS_AllocProcessRecord)

```

LABEL_5:
    v12 = 0;
    goto LABEL_15;
}
v13 = sub_1400C42F4(v10, a2, a4, &v24);
if ( v13 < 0 )
{
    if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 2) != 0 && *((_BYTE *)Process + 41) >= 2u )
    {
        _mm_lfence();
        sub_140008734(*((_QWORD *)Process + 3), 23, &unk_1402C5468, (unsigned int)v13);
    }
}

```

inside CS_AllocProcessRecord: the call to sub_1400C42F4 (CS_GetProcessImagePath) that gathers the new process's image path.

lest jump into sub_1400C42F4 (CS_GetProcessImagePath)

we can see here if the process id is 4 which is system process Return 0xC0000225LL ⇒ STATUS_NOT_FOUND

```

8   if ( a2 == PsInitialSystemProcess )
9       return 0xC0000225LL;

```

PID 4 (the System process) short-circuits with STATUS_NOT_FOUND (0xC0000225) skipped entirely.

now

```

if ( a2 == PsInitialSystemProcess )
    return 0xC0000225LL;
if ( a3 )
{
    v7 = *(PVOID **)(a3 + 56);
    if ( v7 )
    {
        *a4 = v7;
        return 0;
    }
}
*(_OWORD *)P = 0;
v8 = CS_QueryImageName(a2, P);
if ( v8 >= 0 )
{
    *a4 = P;
    if ( (int)CS_NormalizePath(a4) < 0 )
        *a4 = (PVOID *)&unk_1402D08A0;
    if ( P[1] )
        ExFreePoolWithTag(P[1], 0x75427250u);
    return 0;
}
_mm_lfence();
if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 2) != 0 && *((_BYTE *)Process + 41) >= 3u )
{
    ProcessId = PsGetProcessId(a2);
    CS_LogErrorWithPID(*((_QWORD *)Process + 3), 24, &unk_1402C5468, ProcessId, v8);
}
return (unsigned int)v8;
}

```

CS_GetProcessImagePath: fast path grabs FileObject from CreateInfo; slow path calls CS_QueryImageName + CS_NormalizePath.

- Fast path if CreateInfo (a3) exists, grabs the FileObject at offset 56 (the process image .exe path) and returns immediately.
- Slow path fallback if fast path unavailable, calls sub_1403AE624=> ****CS_QueryImageName**** to manually retrieve the image path from KPROCESS (likely via SeLocateProcessImageName).
- On success, sub_1400DAB54 => ****CS_NormalizePath**** validates/normalizes the path; if validation fails, falls back to default descriptor unk_1402D08A0 ("unknown source") rather than failing.
- ExFreePoolWithTag(P[1], 'uBRp') frees the temp buffer used for image name retrieval; prevents memory leak.
- If slow path also fails, logs error (code 24) with the PID via PsGetProcessId and reports to cloud telemetry.

so This function retrieves the process image path via two strategies: fast path from CreateInfo directly, falling back to a manual slow path if unavailable and skips the System process entirely.

2- sub_140377450(CS_DetectionDispatch) ⇒ Detection logic (full treatment in **Section 8**)

Sensor & Engine Checks

- CS_IsSensorEnabled() if non-zero, sensor is disabled; process passes through unchecked.
- if (!g_DetectionEngine) if null, engine not yet initialized (early boot). Logs error 10 and exits. All early-boot processes pass through unmonitored.
- checks g_SensorConfig offset 56 if zero, engine not ready (still loading rules). Exits.
- CS_AcquireEventBuffer(1002) requests process event buffer. If fails → logs error 11, process passes with no detection or telemetry.
- event metadata setup: 2097872 (event flags), 0x10003E9 (event type ID).

```

LOBYTE(v10) = CS_IsSensorEnabled();
if ( !(_BYTE)v10 )
{
    if ( !g_DetectionEngine )
    {
        if ( Process != (PEPROCESS)&Process )
        {
            LODWORD(v10) = *((_DWORD *)Process + 11);
            if ( (v10 & 2) != 0 && *((_BYTE *)Process + 41) >= 2u )
            {
                LOBYTE(v10) = sub_14000870C(*((_DWORD *)Process + 3), 10, &unk_1402C5448);
            }
        }
        LABEL_71:
        v11 = (int)v40;
        if ( v40 )
        {
            v31 = (__int64)v40 + *(int *)(v40[1] + 4LL) + 8;
            LOBYTE(v10) = *((__int64 (__fastcall *))(__int64))(*((_DWORD *)v31 + 8LL))(v31);
            LODWORD(v12) = v35;
            goto LABEL_17;
        }
    }
    LABEL_16:
    LODWORD(v12) = 0;
    goto LABEL_17;
}
LOBYTE(v10) = g_SensorConfig;
if ( !(_BYTE *)g_SensorConfig + 56 )
{
    goto LABEL_17;
}
LODWORD(v10) = CS_AcquireEventBuffer(v12, 1002, &v40);
if ( (int)v10 < 0 )
{
    v11 = (int)Process;
    if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 2 != 0 && *((_BYTE *)Process + 41) >= 5u )
    {
        __mm_lfence();
        LOBYTE(v10) = sub_140008734(*((_DWORD *)Process + 3), 11, &unk_1402C5448, (unsigned int)v10);
    }
    v40 = 0;
    goto LABEL_16;
}
v41 = 0;
LODWORD(v41) = 2097872;
v42 = 0;

```

Red annotations in the image highlight the following checks:

- `CS_IsSensorEnabled()` (line 1)
- `!g_DetectionEngine` (line 4)
- `g_SensorConfig` (line 24)
- `g_SensorConfig + 56` (line 25)
- `CS_AcquireEventBuffer(v12, 1002, &v40)` (line 28)

the early gates: CS_IsSensorEnabled, g_DetectionEngine, g_SensorConfig+56, CS_AcquireEventBuffer any failure passes the process through unchecked.

Creation Path

- CS_AllocPoolBuffer(552) allocates 552 bytes for the creation event structure.
- CS_BuildCreationEvent builds the full event: **PID, parent PID, image path, command line, signing, reputation**, file flags. (Section 2.)

- computes v9 = event object pointer (+32 to reach the primary vtable interface).
- $(*(v9+8))(v9, 72)$ vtable call with 72 = populate/enrich the event with additional data.
- vtable calls with parameters 6 and 10:
 - 6 → likely image hash/fingerprint
 - 10 → likely behavioral signature pointer
 - If both present → calls with 5186 = full detection rule matching (compares against cloud-loaded IOCs and rules).
- if either missing → event cleanup, v9 = 0, logs error 14 (“incomplete data, skipping detection”).

```

DWORD2(v42) = 16778217;
if ( v6 == 1 )
{
    v15 = CS_AllocPoolBuffer(552, v14, 829646160);
    if ( v15 )
        v16 = CS_BuildCreationEvent(
            (unsigned int)v15,
            (unsigned int)v7,
            (unsigned int)a3,
            (unsigned int)g_DetectionEngine,
            (__int64)&v41);
    else
        v16 = 0;
    v10 = v16 + 32;
    v17 = -v16;
    v9 = (_QWORD *) (v10 & -((__int64)(v17 != 0)));
    if ( v9 )
    {
        (*(void (__fastcall **)(__int64, __int64))(*v9 + 8LL))(v10 & -((__int64)(v17 != 0)), 72);
        if ( *(_QWORD *) ( (__int64 (__fastcall **)(__QWORD *, __int64))(*v9 + 8LL))(v9, 6)
            && *(_QWORD *) ( (__int64 (__fastcall **)(__QWORD *, __int64))(*v9 + 8LL))(v9, 10) )
        {
            v21 = 5186;
        }
    }
}

```

the creation path: 552-byte alloc → CS_BuildCreationEvent → vtable selectors 72/6/10, then 5186 for full rule matching when hash and signature both resolved.

Section 2 Building the Event (CS_BuildCreationEvent)

Binary: **CSAgent.sys** · Function: **CS_BuildCreationEvent @ 0x14037889C**

This is the function that actually *builds* the process-creation event handed to the detection engine. Section 1 saw CS_DetectionDispatch allocate 552 bytes and call this to fill it. Now we open it up.

Signature (as Reversed):

```

CS_BuildCreationEvent(a1 = event buffer (552 bytes),
                    a2 = internal process record,
                    a3 = CreateInfo
    (PS_CREATE_NOTIFY_INFO),
                    a4, a5)

```

1. vtable / object setup

First it wires the event up as a COM-like object writes vtable/descriptor pointers into the header:

```

*(a1 + 8) = &unk_1402C3C80;
*(a1 + 40) = &unk_1402C54C0;
*(a1 + 56) = &unk_1402C3D30;
*(a1 + 544) = &off_1402B3030;
sub_14021DE90(a1, 134218483, g_DetectionEngine); // register event, type ID 0x80002F3*(a1) =
off_1402B5688; // primary vtable
*(a1+32)= off_1402B55F0; // secondary interface

```

So the event is a polymorphic object with **4 interface pointers**. `g_DetectionEngine` is the global engine handle same one we keep seeing. `0x80002F3` is the event type ID for “process create”.

```
v5 = (int (__fastcall **)(_QWORD, __int64, _QWORD, _QWORD, __int64))g_DetectionEngine;
*(__QWORD *)(a1 + 8) = &unk_1402C3C80;
*(__QWORD *)(a1 + 40) = &unk_1402C54C0;
*(__QWORD *)(a1 + 56) = &unk_1402C3D30;
*(__QWORD *)(a1 + 72) = &unk_1402C54D8;
*(__QWORD *)(a1 + 544) = &off_1402B3030;
sub_14021DE90(a1, 134218483, v5);
*(__QWORD *)a1 = off_1402B5688;
*(__QWORD *)(a1 + 32) = off_1402B55F0;
```



`CS_BuildCreationEvent` wires the event as a COM-like object: 4 interface pointers + type ID `0x80002F3` (“process create”).

2. pull the core fields from the process record

`a2` is the internal process tracking record (the one `CS_AllocProcessRecord` built). This block copies the interesting stuff into the event:

```
CS_RefProcessRecord(a2); // sub_1400C4784 refcount++ on a2+48
*(a1 + 416) = a2; // keep a back-pointer to the record
*(a1 + 240) = *(a2 + 64); // PID
*(a1 + 336) = *(a2 + 88); // parent PID / session
*(a1 + 296) = CS_ResolveProcessImageInfo(a2); // sub_1400C4DA0 image path
(lazy+cached)
*(a1 + 304) = CS_ResolveProcessCmdInfo(a2); // sub_1400C4DF8 command line
(lazy+cached)
*(a1 + 500) = *(a2 + 212); // flags
```

Ref-count bump first (`CS_RefProcessRecord`) the event holds a reference so the record can’t be freed underneath it while in flight. Image path / command line use a lazy pattern: cached flag set → return cached pointer; else allocate a node, fill, submit telemetry, cache. Computed once, reused.

```
CS_RefProcessRecord(a2);
*(__QWORD *)(a1 + 416) = a2;
*(__QWORD *)(a1 + 240) = *(__QWORD *)(a2 + 64);
*(__DWORD *)(a1 + 336) = *(__DWORD *)(a2 + 88);
*(__QWORD *)(a1 + 296) = CS_ResolveProcessImageInfo(v10);
*(__QWORD *)(a1 + 304) = CS_ResolveProcessCmdInfo(a2);
*(__DWORD *)(a1 + 500) = *(__DWORD *)(a2 + 212);
```



refcount bump, then the core fields copied from the process record into the event: PID, parent PID, image path, command line, flags.

3. token / security context

```
v10 = CS_GetProcessSecurityInfo(a2); // grabs the token object
if ( CS_GetProcessTokenInfo(a2, a1 + 424) < 0 ) // sub_1400C4F78 SID /
integrity
    *(a1 + 424) = 0;
```

```
ProcessSecurityInfo = (void *)CS_GetProcessSecurityInfo((struct _EX_RUNDOWN_REF *)a2);
if ( (int)CS_GetProcessTokenInfo(a2, (volatile signed __int32 **)(a1 + 424)) < 0 )
    *(__QWORD *)(a1 + 424) = 0;
```

`CS_GetProcessTokenInfo` resolves the user SID / integrity level into +424 how the sensor knows who ran the process.

CS_GetProcessTokenInfo walks a rundown-protected token reference and resolves the user **SID / integrity level** into +424. That's how the sensor knows *who* ran the process (SYSTEM vs normal user vs elevated). Then two vtable calls into the engine enrich signing info:

```
(**g_DetectionEngine)(engine, 8, PID, *(a2+128), a1 + 312); // signer / cert
chain
(**g_DetectionEngine)(engine, 9, PID, *(a2+136), a1 + 320); // reputation /
hash
```

Selector **8** = signing/certificate lookup, **9** = reputation/hash. Both keyed by PID + a record field. If either fails, the field is zeroed (event still ships).

```
if ( (**v5)(v5, 8, *(unsigned int *)(a1 + 80), *(_QWORD *)(a2 + 128), a1 + 312) < 0 )
*(_QWORD *)(a1 + 312) = 0;
if ( (**v5)(v5, 9, *(unsigned int *)(a1 + 80), *(_QWORD *)(a2 + 136), a1 + 320) < 0 )
*(_QWORD *)(a1 + 320) = 0;
```

two engine vtable calls: selector 8 = signer/certificate chain, selector 9 = reputation/hash. Failure zeroes the field; event still ships.

4. PID as a string

```
*(a1 + 282) = 18; // UNICODE_STRING max len
*(a1 + 288) = a1 + 256; // buffer inside the event
RtlIntegerToUnicodeString(*(a2 + 64), 10, a1 + 280); // PID -> decimal
string
```

Renders the numeric PID into a decimal Unicode string stored inline at +256. Cloud rules / telemetry like the PID both as int and text.

5. extra string fields (SID string, integrity string)

```
sub_140168140(a1 + 488, &v25); // -> +488 (object) / +496 (byte flag) err
11
sub_140167E00(v10, 0, a1 + 504, 0); // token -> +504
err 12
sub_140167E00(0, *(a2 + 72), a1 + 512, 0); // record -> +512
err 13
```

CrowdStrike never *fails* event construction on a missing sub-field it zeroes it and moves on. Telemetry is best-effort.

6. file/transaction flags (a nice one)

```
v15 = *(a3 + 40); // FileObject from CreateInfo
if ( v15 ) {
    if ( IoGetTransactionParameterBlock(v15) ) *(a1+520) |= 1; // launched inside a TxF
transaction
    if ( (*(a3+40) + 75) ) *(a1+520) |= 2; // FileObject flag bit
    if ( !(*(a3+40) + 77) ) *(a1+520) |= 4; // FileObject flag bit
}
```

- **bit 1** image file is part of an active **transaction** (IoGetTransactionParameterBlock). Classic **process doppelganging** / TxF-launch detection malware maps an image from inside a transaction that later rolls back. Flagged right here.
- **bit 2 / bit 4** two FILE_OBJECT flag bytes (offsets 75 / 77), likely write-access / delete-pending state.

```

v16 = *(struct _FILE_OBJECT **)(a3 + 40);
if ( v16 )
{
    if ( IoGetTransactionParameterBlock(v16) )
        *(_DWORD *)(a1 + 520) |= 1u;
    if ( *(_BYTE *)((_DWORD *) (a3 + 40) + 75LL) )
        *(_DWORD *) (a1 + 520) |= 2u;
    if ( !*(_BYTE *)((_DWORD *) (a3 + 40) + 77LL) )
        *(_DWORD *) (a1 + 520) |= 4u;
}

```

file/transaction flags at +520: bit 1 = image launched inside a TxF transaction (process doppelganging), bits 2/4 = FILE_OBJECT state.

7. final enrichment + name validation

```

if ( *(a1+424) && v10 && (v16 = sub_140025560(), v17(..., v10, a1+296, a1+304, v16,
&v26)) )
    *(a1 + 528) = v26;                                // final verdict/context handle
if ( (*(a2 + 240) & 8) != 0 )
    *(a1 + 528) |= 0x20;                                // record already flagged bit
if ( *(a2 + 248) ) {                                    // short name present?
    *(a1 + 432) = *(a2 + 248);                          // copy UNICODE_STRING
    RtlDuplicateUnicodeString(3, a1+432, a1+472);        // deep-copy the name
    if ( sub_1403AE7AC(a1+472) >= 0 )                    // CS_NormalizePath-style
        validation
            *(a1 + 432) = *(a1 + 472);
}
ObfDereferenceObject(v10);                             // drop the token ref

```

If we have both a token and a resolved SID, a final engine call stores a context handle at +528. Then it validates and **deep-copies** the process short-name so the event owns its own string copy, and drops the token reference from step 3.

```

if ( v24 )
{
    while ( *(_WORD *)((_DWORD *) (a1 + 440) + 2LL * v25) )
    {
        if ( ++v25 >= v24 )
            goto LABEL_66;
    }
    *(_QWORD *) (a1 + 472) = 0;
    *(_WORD *) (a1 + 474) = 0;
    *(_QWORD *) (a1 + 480) = 0;
    if ( RtlDuplicateUnicodeString(3u, (PCUNICODE_STRING) (a1 + 432), (PCUNICODE_STRING) (a1 + 472)) >= 0 )
    {
        _mm_lfence();
        if ( (int)sub_1403AE7AC(a1 + 472) >= 0 )
            *(_QWORD *) (a1 + 432) = *(_QWORD *) (a1 + 472);
    }
    else
    {
        *(_QWORD *) (a1 + 480) = 0;
    }
}
}
}
LABEL_66:
*(_QWORD *) (a1 + 464) = *(_QWORD *) (a2 + 280);
if ( ProcessSecurityInfo )
    ObfDereferenceObject(ProcessSecurityInfo);
return a1;

```

final engine call stores a verdict/context handle at +528, then deep-copies the process short name so the event owns its own string.

Event layout recovered

Offset	Field
+0, +8, +32, +40, +56, +544	vtable / interface pointers
+240	PID (int)
+256	PID as Unicode string (inline buffer)
+296	image path (resolved)
+304	command line (resolved)
+312	signer / certificate info (engine selector 8)
+320	reputation / hash (engine selector 9)
+336	parent PID / session
+416	back-pointer to process record
+424	user SID / integrity
+432	process short name (owned copy)
+488 / +496	extra string object + flag
+500	process flags
+504	token-derived string
+512	record-derived string
+520	file/transaction flags (bit1 = TxF/doppelganging)
+528	final verdict / context handle

CS_BuildCreationEvent is the sensor's full **process-creation telemetry record**: identity (PID/parent/name), provenance (image path, command line, TxF flag), trust (signer, reputation, SID/integrity) everything the cloud rules need.

Section 3 The Kernel Eyes (every callback source)

Binary: CSAgent.sys

csagent.sys registers **six** kernel notification sources. This is the entire “where detection happens” surface:

#	Source	Register API	Callback fn	Sees
1	Process	PsSetCreateProcessNotifyRoutineEx	CS_ProcessNotifyCallback	process create / exit / block
2	Thread	PsSetCreateThreadNotifyRoutine(Ex)	CS_ThreadCreateCallback	thread create/exit, remote threads
3	Image	PsSetLoadImageNotifyRoutine	CS_LoadImageCallback	EXE/DLL/ driver loads
4	Object	ObRegisterCallbacks	CS_ObPreOpCallback / CS_ObPostOpCallback	handle opens to process/thread (LSASS)
5	Registry	CmRegisterCallbackEx	CS_RegistryCallback	registry read/write/delete
6	File	FltRegisterFilter	(minifilter Section 5)	file create/write/delete

Plus **ETW output**: EtwRegister / EtwWriteTransfer telemetry to CSFalconService.exe. All funnel into the same event object + engine.

1. Process (recap)

Covered in Sections 1–2. One-line walk: CS_ProcessNotifyCallback (0x1400C3310) → CS_ProcessEventHandler (0x1400C3320) → CS_DetectionDispatch (0x140377450) → CS_BuildCreationEvent (0x14037889C) → rule engine selector 72. **Collected**: PID, PPID, image path, command line, SID/integrity, signer, reputation, TxF flag. **Kill**: negative NTSTATUS → CreateInfo->CreationStatus (+64) → process aborted.

2. Thread creation (remote-thread / injection detection)

Where CrowdStrike catches **CreateRemoteThread**, thread hijacking, injected APC-style threads.

Walk: CS_RegisterThreadCallback (0x1400D0E34) tries the **Ex** variant first via a dynamically-resolved pointer (qword_1402FB800), falls back to plain PsSetCreateThreadNotifyRoutine. Then CS_ThreadCreateCallback (0x1400CFE50) the meat.

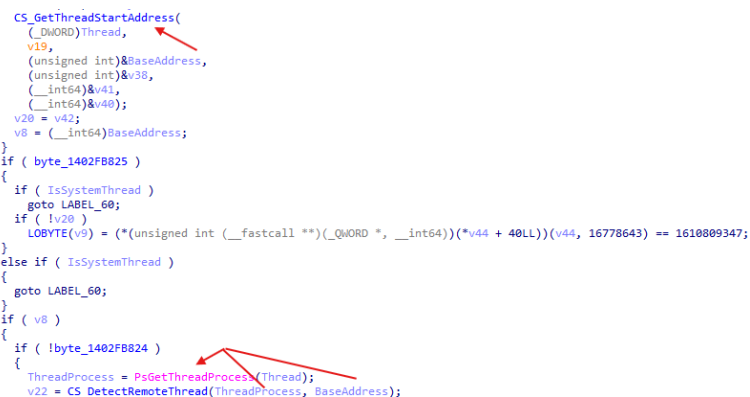
```

CS_ThreadCreateCallback(ProcessId, ThreadId, Create):
    if (g_TraceEnabled) CS_TraceEnter(...)           // ETW trace begin
    if (KeGetCurrentIrql() != 0) bail                // must be PASSIVE_LEVEL
    if (KeAreAllApcsDisabled()) bail                 // APCs must be enabled
    CS_AcquireEventBuffer(1002)

    if (Create):                                     // — THREAD CREATE —
        PsLookupThreadByThreadId(ThreadId, &Thread)
        IsSystemThread = PsIsSystemThread(Thread)
        if (CS_IsExcludedProcess(cur, curPid, 0x5000023)) bail // self/allowlist
        CS_GetThreadStartAddress(Thread, &startAddr,...) // Win32StartAddress
        if (!IsSystemThread && startAddr && !byte_1402FB824):
            ThreadProcess = PsGetThreadProcess(Thread)
            CS_DetectRemoteThread(ThreadProcess, startAddr) // * remote-thread
check
    evt = ExAllocatePoolWithTagPriority(0x168, 'Tes1') // 360-byte thread
event
    CS_BuildThreadCreateEvent(evt, ProcessId, ThreadId, startAddr, ...)
    submit(evt)
    else:                                           // — THREAD EXIT —
        if (CS_IsExcludedProcess(cur, curPid, 36)) bail
        evt = ExAllocatePoolWithTagPriority(0x168, 'Tes2') // termination event
        CS_BuildThreadTermEvent(evt, ProcessId, ThreadId)
        submit(evt)

```

CS_DetectRemoteThread compares the **thread's owning process** against the **creator context**. A thread whose start address lands in a *different* process than the caller = classic CreateRemoteThread injection. Fires only when: not a system thread, has a resolved start address, and byte_1402FB824 (config flag) is set.



```

CS_GetThreadStartAddress(
    (_DWORD)Thread,
    v19,
    (unsigned int)&BaseAddress,
    (unsigned int)&v38,
    (__int64)&v41,
    (__int64)&v40);
v20 = v42;
v8 = (__int64)BaseAddress;
}
if ( byte_1402FB825 )
{
    if ( IsSystemThread )
        goto LABEL_60;
    if ( !v20 )
        LOBYTE(v9) = (*(unsigned int (__fastcall *)(_QWORD *, __int64))(*v44 + 40LL))(v44, 16778643) == 1610809347;
}
else if ( IsSystemThread )
{
    goto LABEL_60;
}
if ( v8 )
{
    if ( !byte_1402FB824 )
    {
        ThreadProcess = PsGetThreadProcess(Thread);
        v22 = CS_DetectRemoteThread(ThreadProcess, BaseAddress);
    }
}

```

CS_ThreadCreateCallback entry: PASSIVE_LEVEL / APC guards, then the thread-create vs thread-exit split.

```

PoolWithTagPriority = ExAllocatePoolWithTagPriority((POOL_TYPE)512, 0x168u, 0x32736554u, NormalPoolPriority);
if ( PoolWithTagPriority )
    v31 = CS_BuildThreadTermEvent(PoolWithTagPriority, ProcessId, ThreadId);
else

```

CS_DetectRemoteThread: start address in a different process than the caller = classic CreateRemoteThread injection.

```

LABEL_60:
v23 = ExAllocatePoolWithTagPriority((POOL_TYPE)512, 0x168u, 0x31736554u, NormalPoolPriority);
if ( v23 )
{
    _mm_lfence();
    v24 = CS_BuildThreadCreateEvent(
        (_DWORD)v23,
        ProcessId,
        (_DWORD)ThreadId,
        (_DWORD)v23,
        (_int64)BaseAddress,
        v38,
        v41,
        v40,
        v42,
        IsSystemThread == 0,
        v9,
        v39);
}

```

the 360-byte thread event ('Tes1') built and submitted with ProcessId, ThreadId and the resolved start address.

3. Image / DLL / driver load

Walk: CS_RegisterImageCallback (0x1400B13E0) → CS_LoadImageCallback (0x1400B03A0).

```

CS_LoadImageCallback(FullImageName, ProcessId, ImageInfo):
    if ((ImageInfo->Properties & 0x400) == 0) bail // need extended IMAGE_INFO
    isKernelImage = (ImageInfo->Properties >> 8) & 1 // SystemModeImage bit
    CS_AcquireEventBuffer(1002)
    if (isKernelImage): // — DRIVER LOAD —
        CS_HandleDriverLoad(pid, pid, FileObject, characteristics)
    else: // — USER IMAGE —
        nt = RtlImageNtHeader(imageBase) // parse PE
        collect: Characteristics, DllCharacteristics, Subsystem, Machine,
        SubsystemVersion
        if (imageBase == PsGetProcessSectionBaseAddress(cur) // MAIN EXE
            && !KeAreAllApcsDisabled() && !KeIsAttachedProcess()):
            CS_HandleMainImageLoad(...)
        CS_DispatchImageEvent(qword_1402FB7A0, pid, FileObject, characteristics) // ★
event
    CS_SubmitImageDetection(engineCtx, {2, curPid, curTid, ProcessId})

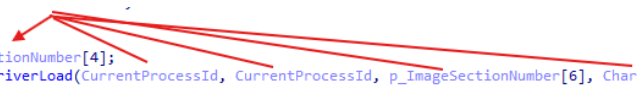
```

- **Driver loads are first-class** (CS_HandleDriverLoad) this is BYOVD (bring-your-own-vulnerable-driver) visibility. Separate path from user images.
- **Main-image detection** compares loaded image base to PsGetProcessSectionBaseAddress correlates image-load to process-create.
- **PE header parsed in kernel** (RtlImageNtHeader) → characteristics/subsystem/machine become telemetry. Malformed PE = interesting fuzz target.

```

if ( v12 )
{
    v35 = p_ImageSectionNumber[4];
    v24 = CS_HandleDriverLoad(CurrentProcessId, CurrentProcessId, p_ImageSectionNumber[6], Characteristics);
}
else

```



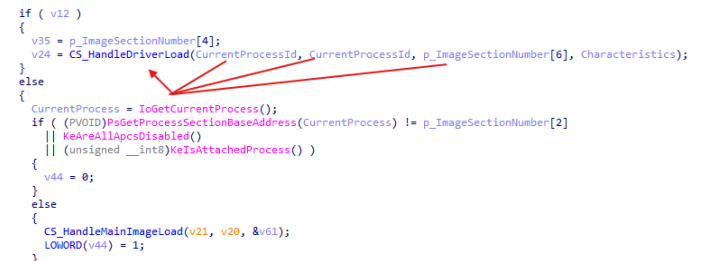
RtlImageNtHeader parses the PE in kernel; Characteristics / DllCharacteristics / Subsystem / Machine become telemetry (a fuzz surface).

```

v55 = RtlImageNtHeader(p_ImageSectionNumber[2]);
Characteristics = v55->FileHeader.Characteristics;
v41 = Characteristics;
v39 = Characteristics;
DllCharacteristics = v55->OptionalHeader.DllCharacteristics;
v40 = DllCharacteristics;
Subsystem = v55->OptionalHeader.Subsystem;
v46 = *(_DWORD *)&v55->OptionalHeader.MajorSubsystemVersion;
Machine = v55->FileHeader.Machine;
if ( v12 )
{
    v35 = p_ImageSectionNumber[4];
    v24 = CS_HandleDriverLoad(CurrentProcessId, CurrentProcessId, p_ImageSectionNumber[6], Characteristics);
}

if ( v12 )
{
    v35 = p_ImageSectionNumber[4];
    v24 = CS_HandleDriverLoad(CurrentProcessId, CurrentProcessId, p_ImageSectionNumber[6], Characteristics);
}
else
{
    CurrentProcess = IoGetCurrentProcess();
    if ( (PV0ID)PsGetProcessSectionBaseAddress(CurrentProcess) != p_ImageSectionNumber[2]
        || KeAreAllApcsDisabled()
        || (unsigned __int8)KeIsAttachedProcess() )
    {
        v44 = 0;
    }
    else
    {
        CS_HandleMainImageLoad(v21, v20, &v61);
        LOWORD(v44) = 1;
    }
}

```



kernel images take a first-class branch (*CS_HandleDriverLoad*) bring-your-own-vulnerable-driver visibility.

4. Object callbacks (LSASS / credential-theft shield)

The single most security-relevant callback what stops **mimikatz-style** **OpenProcess(PROCESS_VM_READ, lsass)** dumping.

Walk: **CS_RegisterObjectCallbacks** (0x1400C1DE8) registers pre-op **CS_0bPreOpCallback** + post-op **CS_0bPostOpCallback** for both **PoProcessType** and **PoThreadType**. → **CS_0bPreOpCallback** (0x1400C20D0) → **CS_CheckHandleAccess** (0x1400C2234), the actual decision.

```

CS_0bPreOpCallback(reg, opInfo):
    if ( ExGetPreviousMode() // request came from USER
MODE
        && (opInfo->Flags & 1) == 0 // real op (not kernel
handle)
        && ( opInfo->Operation is CREATE(1)/DUPLICATE(2)
        && !CS_CheckHandleAccess(ObjectType, TargetProcess, DesiredAccess) ) ):
        reg->callback(reg, opInfo) // → strip/deny logic

CS_CheckHandleAccess(type, targetProc, access):
    if (type == PoProcessType) harmless = (access & 0xFFEDCFFF) == 0;
    if (type == PoThreadType) harmless = (access & 0xFFEDF3FF) == 0;
    if (harmless) return 1; // allow, no stripping
    if (CS_GetHandlePolicy(...) ok && policy < 0x3000):
        if (targetProc == IoGetCurrentProcess() // opening a handle to
YOURSELF
            || threadOwner == PsGetCurrentProcessId()):
            return 1; // self-access always
allowed
    return 0; // → strip access

```

- Process mask 0xFFEDCFFF cleared bits (~mask = 0x00123000) are the only “harmless” ones. **PROCESS_VM_READ/WRITE/OPERATION, CREATE_THREAD, DUP_HANDLE, TERMINATE** all trip the check.
- Thread mask 0xFFEDF3FF same for **THREAD_SET_CONTEXT, SUSPEND_RESUME**, etc.
- **Self-access exception** a process opening a handle to itself is always allowed. The escape hatch every EDR needs.

```

{
if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 2) != 0 && *((_BYTE *)Process + 41) >= 5u )
sub_14000878C(*((_DWORD *)Process + 3), 19, &unk_1402C5428, a1);
if ( ExGetPreviousMode()
&& *((_DWORD *)a2 + 4) & 1) == 0
&& *((_DWORD *)a2 != 1 && *((_DWORD *)a2 != 2
|| !(unsigned __int8)CS_CheckHandleAccess(*(_DWORD *)a2 + 16), *(_DWORD *)a2 + 8, **(unsigned int **)a2 + 32))) )
{
*(void (__fastcall **)(__int64, __int64))(*(_DWORD *)a1 + 24LL))(a1, a2);
}
return 0;
}

```

CS_CheckHandleAccess: process mask 0xFFEDCFFF / thread mask 0xFFEDF3FF decide "harmless"; VM_READ/WRITE, CREATE_THREAD, DUP_HANDLE trip the check. Self-access always allowed.

Blind spots: the entire filter is gated on `ExGetPreviousMode() == UserMode` a **kernel-mode** actor opening a handle is not filtered here. `opInfo->Flags & 1` (kernel handles) skip it. Altitude ordering games. Static masks.

5. Registry

Catches persistence (Run keys), defense tampering, credential access via hives.

Walk: `CS_RegisterRegistryCallback (0x1400C8F78) → CS_RegistryCallback (0x1400C7790) → CS_RegistryFilterDispatch (0x1400C7CA8).`

```

CS_RegistryCallback(ctx, NotifyClass, Arg2):
    if (ExGetPreviousMode() == KernelMode):           // kernel-origin op
        if (NotifyClass != 47) return 0;              // only RegNtPreDeleteValueKey
    inspected
    else:                                              // user-origin op
        KeEnterCriticalRegion()
        swap = KeSetKernelStackSwapEnable(0)         // pin kernel stack (anti-recursion)
        if (!recursionGuardSet):
            recursionGuard = 1
            result = CS_RegistryFilterDispatch(ctx, NotifyClass, Arg2) // *
            recursionGuard--

```

- **User-mode ops fully inspected; kernel-mode almost entirely ignored** only
RegNtPreDeleteValueKey (class 47) is looked at from kernel. Same `ExGetPreviousMode()` trust boundary.
- **Recursion guard** prevents the sensor's own registry access from re-entering while set, that thread's ops aren't re-inspected.

```

if ( !ExGetPreviousMode() )
{
v6 = 1;
if ( (_DWORD)Argument1 != 47 )
return 0;
v7 = qword_1402F8318;
CurrentThreadId = PsGetCurrentThreadId();
if ( (int)sub_1401BF8A4(v7, 6, CurrentThreadId, &v11) < 0 || !v11 )
return 0;
}

```

CS_RegistryCallback: kernel-origin ops ignored except RegNtPreDeleteValueKey (class 47); user-origin ops fully inspected.

```

KeEnterCriticalRegion();
sub_1401BF54C(v13, (unsigned int)v12, 40);
sub_1401BF5A0((unsigned int)v12, v13, (char *)&v12 + 8, (char *)&v12 + 4);
BYTE5(v12) = KeSetKernelStackSwapEnable(0) != 0;
if ( *((_BYTE *)((_DWORD *)v12 + 1) + 32LL) )
{
v10 = 0;
}

```

the recursion-guarded dispatch: KeSetKernelStackSwapEnable(0) pins the stack, guard blocks the sensor's own registry access from re-entering.

Section 4 The Network Inspector (WFP)

Binary: **CSAgent.sys**

csagent.sys embeds a full **Windows Filtering Platform (WFP)** engine connection visibility, DNS/telemetry, and the ability to **kill C2 traffic**. Imports: fwpmclnt.sys (53 fns) + NETIO.SYS.

The WFP building blocks

Plane	APIs	Purpose
Management	FwpmEngineOpen0, FwpmProviderAdd0, FwpmSubLayerAdd0, FwpmFilterAdd0, FwpmCalloutAdd0	install provider/sublayers/filters
Callout (data)	FwpsCalloutRegister2	the classify functions that inspect each packet
Flow context	FwpsFlowAssociateContext0, FwpsFlowRemoveContext0, FwpsFlowAbort0	per-connection state
Stream DPI	FwpsCopyStreamDataToBuffer0, FwpsCloneStreamData0	deep-inspect TCP payloads (DNS, HTTP, TLS SNI)
Injection	FwpsInjectTransportSendAsync1, FwpsStreamInjectAsync0, FwpsInjectionHandleCreate0	inject/modify/reset packets connection-kill
Sockets (WSK)	WskRegister, GetIpForwardTable2, GetIpNetTable2	the sensor's own kernel network client + topology

Not just a monitor provider + sublayers + filters + callouts + injection + its own socket stack. Firewall-grade.

1. Two filtering engines

Walk: CS_RegisterCallouts_Eng1 (0x14020DE50) and CS_RegisterCallouts_Eng2 (0x14021E738) near-identical loops registering callouts against two engine handles:

- g_WfpEngine1 (0x1402FB328) Eng1 handles layer indices **0–5**
- g_WfpEngine2 (0x1402FB370) Eng2 a bitmask of layers (**0,1,3,4,6**)

Both draw callout descriptors from the same global table (g_CalloutTable 0x1402F9A48, g_CalloutTableCount 0x1402F9A40), built by 11 per-layer functions.

```

v4 = FwpsCalloutRegister2(g_WfpEngine1, v14, (char *)v8 + 36);
if ( v4 >= 0 )
{
  *((_BYTE *)v8 + 44) = 1;
  mm_lfence();
  if ( (_DWORD)v2 == 4 )
  {
    v10 = (_QWORD *)v8[3];
    if ( *v10 == 0x49E4C1703B89653CLL && v10[1] == 0x3E9AE1EEEE0CDB1LL )
      *(_DWORD *)(a1 - 88) = *v9;
  }
  else
  {
    v11 = (_QWORD *)v8;
    if ( *(_QWORD *)v8 == 0x4838FD4EC5996155LL && v11[1] == 0xCCB1ACA3FBA4A784uLL )
    {
      *(_DWORD *)(a1 - 112) = *v9;
    }
    else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0x9C7A073B1630E084uLL )
    {
      *(_DWORD *)(a1 - 108) = *v9;
    }
    else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0x841C3CF536B1C787uLL )
    {
      *(_DWORD *)(a1 - 104) = *v9;
    }
    else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0xDAA5B22F317859BCuLL )
    {
      *(_DWORD *)(a1 - 100) = *v9;
    }
    else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0x36B587E8707E75AELL )
    {
      *(_DWORD *)(a1 - 96) = v7[12 * v6 + 9];
    }
  }
}

```

CS_RegisterCallouts_Eng1/Eng2: near-identical loops registering callouts against the two WFP engine handles from a shared callout table.

The callout GUIDs (a real fingerprint)

prefix qword 0x4838FD4EC5996155 (shared by most callouts)
 + 0xCCB1ACA3FBA4A784 / 0x9C7A073B1630E084 / 0x841C3CF536B1C787 ...
 one different family: 0x49E4C1703B89653C / 0x3E9AE1EEEE0CDB1 (layer index 4)

Stable **host/network fingerprint** netsh wfp show state and match these keys.

```

v11 = (_QWORD *)v8;
if ( *(_QWORD *)v8 == 0x4838FD4EC5996155LL && v11[1] == 0xCCB1ACA3FBA4A784uLL )
{
  *(_DWORD *)(a1 - 112) = *v9;
}
else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0x9C7A073B1630E084uLL )
{
  *(_DWORD *)(a1 - 108) = *v9;
}
else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0x841C3CF536B1C787uLL )
{
  *(_DWORD *)(a1 - 104) = *v9;
}
else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0xDAA5B22F317859BCuLL )
{
  *(_DWORD *)(a1 - 100) = *v9;
}
else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0x36B587E8707E75AELL )
{
  *(_DWORD *)(a1 - 96) = v7[12 * v6 + 9];
}
else if ( *v11 == 0x4838FD4EC5996155LL && v11[1] == 0xABE9A86122464499uLL )
{
  *(_DWORD *)(a1 - 92) = v7[12 * v6 + 9];
}

```

the callout GUIDs (shared prefix 0x4838FD4EC5996155) a stable host/network fingerprint you can match in netsh wfp show state.

2. Connection tracking CS_FlowEstablishClassify (0x1400B5DF0)

When the ALE “flow established” layer fires, this builds a **per-flow tracking record**:

```

CS_FlowEstablishClassify(adapterCtx, layerData, numLayers, flowId, ...):
    if (adapterCtx->flowCount < 0x2710) { // cap: 10,000 concurrent
flows
        rec = CS_AllocPoolSized(NonPaged, size, 'CSFR'); // ← CrowdStrike Flow
Record
        rec->flowId = flowId; rec->timestamp = sub_140149C04();
        memcpy(rec->tuple, tupleData, tupleLen); // the 5-tuple
        for each layer:
            FwpsFlowAssociateContext0(flowId, layerId, calloutId, (UINT64)rec); // ★
attach
        link rec into adapterCtx->flowList;
    }

```

- **FwpsFlowAssociateContext0** attaches the record so **every packet/stream chunk on that connection is delivered to the callouts *with this record as context*** that's what makes inspection *stateful*.
- **Pool tag 'CSFR'** (0x52465343) !poolused shows one per tracked connection. The **10,000-flow cap** is a hard ceiling: beyond it, new flows aren't tracked.

```

if ( *(_DWORD *) (a1 + 808) < 0x2710u )
{
    v12 = (volatile __int32 *) CS_AllocPoolSized(0, a7 + 103, 0x52465343u);
    v13 = v12;
    if ( v12 )
    {
        *(_WORD *) v12 + 22 = a3;
        _InterlockedExchange(v12 + 12, 0);
        *(_QWORD *) v12 + 7 = a4;
        *(_DWORD *) v12 + 10 = *a5;
        *(_WORD *) v12 + 32 = 0;
        v15 = sub_140149C04();
        v16 = 0;
        *(_QWORD *) v13 + 9 = v15;
        for ( *(_DWORD *) v13 + 23 = 0; v16 < *(_WORD *) v13 + 22; v13[v18 + 5] = *(_DWORD *) (a2 + 12 * v17 + 4) )
        {
            v17 = v16++;
            v18 = 3 * v17;
            LOWORD(v13[v18 + 4]) = *(_WORD *) (a2 + 12 * v17);
        }
    }
}

```

CS_FlowEstablishClassify allocates a per-flow record (pool tag 'CSFR') and stores the 5-tuple capped at 10,000 concurrent flows.

```

if ( a3 )
{
    v20 = v13 + 5;
    do
    {
        v21 = FwpsFlowAssociateContext0(a4, *(_WORD *) v20 - 2, *v20, (UINT64) v13);
        if ( v21 )
        {
            if ( Process != (PEPROCESS) & Process
                && *(_DWORD *) Process + 11 & 0x400000 != 0
                && *(_BYTE *) Process + 41 >= 2u )
            {
                sub_14001351C(*(_QWORD *) Process + 3, 65, &unk_1402C4150, *((unsigned __int16 *) v20 - 2), *v20, v21);
            }
            v14 = v21;
        }
        else
        {
            ...
        }
    }
}

```

the FwpsFlowAssociateContext0 loop that attaches the record to every layer what makes packet inspection stateful per-connection.

Because the record is associated as flow context, the stream/transport classify callbacks receive it on every packet accumulating per-connection state (bytes, direction, timing), correlating to the owning **process** (PID from the ALE layers), and handing payloads to DPI *with* connection identity attached. Connection-oriented, not stateless packet filtering.

3. The action side injection & connection kill

Walk: CS_InjectTransportPacket (0x1401D4260) the transport/stream injector.

```
CS_InjectTransportPacket(...flowId, proto, ...):
    buf = ExAllocatePoolWithTagPriority(NonPaged, size, 'pPDd') // tag PdPp
    CS_MemCopy(buf, payload, size)
    mdl = IoAllocateMdl(buf, size); MmBuildMdlForNonPagedPool(mdl)
    FwpsAllocateNetBufferAndNetBufferList0(..., mdl, ..., &nbl)
    if (proto == 6 /* TCP */):
        FwpsStreamInjectAsync0(g_InjHandleStream, ..., flowId, ..., nbl, size, ...) // TCP stream
    else:
        FwpsInjectTransportSendAsync1(...) // transport
    datagram
```

Inject a **TCP RST/FIN** into a flagged flow → connection dies (C2 kill). Or inject/replace stream data → sinkhole.

```
if ( a10 == 6 )
{
    _mm_lfence();
    if ( a9 )
        v28 = a8 & 0xFFFE | 1;
    else
        v28 = a8 & 0xFFFFE0000 | 0x10000;
    v29 = *(_QWORD *)(a1 + 848);
    if ( v29 )
        v30 = *(_DWORD *)(v29 + 72);
    else
        v30 = 0;
    v23 = FwpsStreamInjectAsync0(
        qword_1402FB940,
        0,
        0,
        flowId,
        v30,
        0x14u,
        v28,
        netBufferList,
        a3,
        sub_1401D4130,
        completionContext);
}
```

CS_InjectTransportPacket: copies payload into a 'pPDd' buffer, builds the MDL + NET_BUFFER_LIST for injection.

```
if ( a9 )
{
    _mm_lfence();
    v33 = FwpsInjectTransportReceiveAsync0(
        qword_1402FB928,
        0,
        0,
        0,
        2u,
        compartmentId,
        interfaceIndex,
        subInterfaceIndex,
        netBufferList,
        sub_1401D4130,
        completionContext);
}
else
{
    FirstNetBuffer = netBufferList->FirstNetBuffer;
    P = (UCHAR *)ExAllocatePoolWithTagPriority((POOL_TYPE)512, 4u, 0x61724450u, NormalPoolPriority);
    v18 = (FWPS_TRANSPORT_SEND_PARAMS1 *)ExAllocatePoolWithTagPriority(
        (POOL_TYPE)512,
        0x30u,
        0x64734450u,
        NormalPoolPriority);
    v35 = ExAllocatePoolWithTagPriority((POOL_TYPE)512, 0x10u, 0x63744450u, NormalPoolPriority);
}
```

the protocol branch: proto == 6 (TCP) → FwpsStreamInjectAsync0, else FwpsInjectTransportSendAsync1 the connection-kill primitive.

Killing tracked connections CS_KillFlaggedFlows (0x1401D108C)

When the engine flags flows for termination:

```
CS_KillFlaggedFlows(ctx):
    for (bucket = 0; bucket < flowMgr->count(); bucket++) {
        collect up to 1000 flow ids;                // bounded batch (0x3E8)
        for each collected flow:
            CS_ResetFlow(ctx, flow, proto=6 /*TCP*/); // → CS_InjectTransportPacket
    }
(RST)
}
```

CS_ResetFlow (0x1401D626C) calls the injection primitive to inject a **TCP reset** into the flow. The application just sees its connection drop. That's the mechanical answer to “how does CrowdStrike kill C2 traffic”: it doesn't ask the firewall to block it **injects a reset into the live flow it's been tracking**.

```
v16 = CS_ResetFlow(a1 + 520, v22, v7, v21);
if ( v16 >= 0 )
{
    v17 = v21[0];
}
else
{
    if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 1) != 0 && *((_BYTE *)Process + 41) >= 2u )
    {
        _mm_lfence();
        sub_1400088F8(*((_QWORD *)Process + 3), 52, &unk_1402D8000, v22, v16);
    }
    v17 = 2;
    v21[0] = 2;
}
if ( v17 == 1 )
    CS_InjectTransportPacket(a1, 0, 0, 0, 0, v20, *v14, 589824, 0, 6, 0, 0, UNSPECIFIED_COMPARTMENT_ID, 0, 0);
sub_1401D49C0(a1, *v14);
if ( v1 >= 0 )
    v1 = v16;
++v15;
++v14;
--v13;
}, ...
```

CS_KillFlaggedFlows walks tracked flows in bounded batches and calls CS_ResetFlow → injects a TCP reset. This is how C2 gets killed.

4. The sensor's own network stack (WSK)

csagent is also a WSK **client** (wskRegister), plus NETIO topology calls (GetIpForwardTable2, GetIpNetTable2, GetIfEntry2). It can open its own kernel sockets (cloud comms / sinkhole from ring-0) and enumerate/watch network topology (interfaces, routes, ARP) noticing rogue VPNs or new adapters.

Network artifacts: WFP provider + 2 sublayers + many callouts (netsh wfp show state), pool tags CSFR (tracked flows) and PdPp (injected buffers), injection handles, WSK registration.

Section 5 The File System Minifilter (FLTMGR)

Binary: CSAgent.sys

The last major detection class file **create/open, write, delete, rename, execution-mapping**. Core of ransomware detection, dropper detection, file-based IOC matching.

1. Registration CS_RegisterMinifilter (0x1403630B0)

```
CS_RegisterMinifilter(state):
    Registration = { ..., OperationRegistration = 0x1402B4880, FilterUnload = CS_FltUnload,
... }
    status = FltRegisterFilter(DriverObject, &Registration, &Filter);
    if ( status >= 0 ) {
        ExInitializeRunDownProtection(&RunRef);
        sub_140380440(6, Filter);          // start filtering / attach instances
    }
```

Runs under a critical region + state machine (dword_140515008: 1=unregistered, 2=registering, other=active) so registration can't race.

```
((void __fastcall __cdecl) __cdecl) sub_1403630B0(v4, v3);
KeLeaveCriticalRegion();
v4 = FltRegisterFilter*((PDRIVER_OBJECT *)Data + 11), (const FLT_REGISTRATION *)Registration, &Filter);
if ( v4 >= 0 )
{
    if ( v3 )
    {
        ExInitializeRunDownProtection(&RunRef);
        sub_140380440(6, Filter);
    }
}
```

the minifilter registration, guarded by a state machine

2. The operation table what's watched

The OperationRegistration array at 0x1402B4880 hooks **12 IRP major functions**:

IRP major	Pre-op	Why CrowdStrike cares
IRP_MJ_CREATE	CS_FltPreCreate	every file open/create path, access, disposition
IRP_MJ_SET_INFORMATION	CS_FltPreSetInfo	rename + delete ransomware trips this
IRP_MJ_QUERY_INFORMATION	CS_FltPreQueryInfo	metadata queries
IRP_MJ_CLEANUP	CS_FltPreCleanup	last handle closed
IRP_MJ_READ	CS_FltPreRead	reads (exfil signals)
IRP_MJ_WRITE	CS_FltPreWrite	writes ransomware/dropper content
IRP_MJ_SET_SECURITY	CS_FltPreSetSecurity	ACL tampering
IRP_MJ_ACQUIRE_FOR_SECTION_SYNC (0xFF)	CS_FltPreAcquireForSection	file mapped for execution
IRP_MJ_DIRECTORY_CONTROL	CS_FltPreDirControl	directory enumeration
IRP_MJ_DEVICE_CONTROL	CS_FltPreDeviceControl	IOCTLs

IRP major	Pre-op	Why CrowdStrike cares
(0xF2 fast-io)	FsControl-adjacent	FS-control fast path
IRP_MJ_FILE_SYSTEM_CONTROL	CS_FltPreFsControl	FSCTLs (reparse, USN)

Near-total file-system coverage. Most security-relevant: SET_INFORMATION (delete/rename) and ACQUIRE_FOR_SECTION (about to execute/memory-map complements the image-load callback).

```

.rdata:000000001402B4878 align 20h ; SUB_1402B4878
.rdata:000000001402B4880 unk_1402B4880 db 0 ; DATA XREF: .rdata:off 1402B47E0fo
.rdata:000000001402B4881 db 0
.rdata:000000001402B4882 db 0
.rdata:000000001402B4883 db 0

```

12 IRP majors hooked near-total file-system visibility

3. The callback pattern thin trace shim → real handler

Every pre-callback is the same shape (here CS_FltPreCreate, 0x1403622A0):

```

CS_FltPreCreate(Data):
    if (g_TraceEnabled & 2) trace_enter(...); // ETW/debug, gated
    result = CS_HandleFileCreate(Data); // ← the real work
    if (g_TraceEnabled & 2) trace_exit(..., result);
    return result; // FLT_PREOP_*
verdict

```

Real logic lives in per-op handlers: CS_FltPreCreate → CS_HandleFileCreate (0x1400A2834); CS_FltPreSetInfo → CS_HandleFileSetInfo (0x14009E2F4, delete/rename); CS_FltPreDeviceControl → CS_HandleFileDeviceControl (0x1400A7190). Note g_TraceEnabled & 2 same global as the network layer (& 1), different bit.

```

if ( (g_TraceEnabled & 2) != 0 )
{
    LOBYTE(Flags) = Data->Iopb->MajorFunction;
    sub_140095CF4((DWORD)Data, (unsigned int)&unk_1402B3000, 0, Flags, IrpFlags, Data->Flags, Data->IoStatus.Status);
}
v5 = CS_HandleFileCreate(Data);
v7 = v5;
if ( (g_TraceEnabled & 2) != 0 )
{
    LOBYTE(v6) = MajorFunction;
    sub_140095D80(Data->Flags, (unsigned int)&unk_1402B2938, 0, v6, IrpFlags, Data->Flags, Data->IoStatus.Status, v5);
}
return v7;
}

```

every FLT callback is a thin trace shim around the real handler same pattern as the rest of the sensor

The handler resolves the **normalized file path** (via FLT_FILE_NAME_INFORMATION), determines the *kind* of set-info (rename/delete/hardlink), packages it → shared event ring → engine → cloud.

File artifacts: registered minifilter + altitude (fltmc filters), per-volume instances (fltmc instances), ETW/debug trace (bit & 2, usually off).

Section 6 cspcm4.sys, the Pinned Code Module

Binary: **cspcm4.sys** (imagebase 0x180000000)

Section 1 saw csagent import 6 functions from **cspcm4.sys** “anti-tamper and sensor code integrity.” Time to open it. It’s tiny **53 KB, 101 functions** so we can Reverse essentially all of it.

Verdict up front: cspcm4 is **not** a scanner. It’s a **shared-services broker** a resident (“pinned”) module providing a small stable ABI of trusted primitives (memory, callback registry, pinned code page, APC injection) that the rest of the driver family rendezvous with. Kernel glue.

```
module:      cspcm4.sys   image size 0xD000 (53 KB)   101 functions
exports:     ordinals 800–806 + DllInitialize(807) + DllUnload(808)
imports:     ntoskrnl only  Ex/Ke/Rtl/Mm. No FLTMGR, no crypto, no network.
```

1. DllInitialize the rendezvous

Walk: DllInitialize (0x180001470). The whole design in one function.

```
DllInitialize():
    CS_InitSecurityCookie();
    if ( KUSER_SHARED_DATA[0x2EC] ) return 0xC000041F;    // boot/debug-state gate:
abort
    ObjectAttributes = { name, OBJ_CASE_INSENSITIVE|OBJ_PERMANENT };
    if ( ExCreateCallback(&cb, &oa, Create=0, Allow=0) < 0 )    // try to OPEN
existing
    {
        // ---- FIRST loader: we own the module ----
        qword_180005168 = &shared_context;
        CS_PcmInit();
        if ( KUSER_SHARED_DATA[0x330] ) shared[4424] = 0;    // skip pinned-page copy
        else {
            CS_MemMove(shared+328, <current 4KB page>, 4096); // ← COPY THE PINNED
PAGE
            shared[4424] = 1;
        }
        ExCreateCallback(&cb, &oa, Create=1, Allow=0);
        ExRegisterCallback(cb, Pcm_BrokerCallback, 0);        // publish ourselves
    }
    else
        ExNotifyCallback(cb, &qword_180005168, 0);            // hand context to the
owner
```

cspcm4 uses a **named Ex callback object as a singleton rendezvous**. First CS module to init creates the object and publishes a broker callback; later modules just *notify* the existing object and exchange context pointers. Pcm_BrokerCallback (0x180001010) is trivial: *Argument1 = qword_180005168 whoever calls gets a pointer to the shared context. Classic way for a driver family to share one trusted state block without exporting it by symbol.

```

if ( ExCreateCallback(&CallbackObject, &ObjectAttributes, 0, 0) < 0 )
{
    dword_180005170 = 2;
    qword_180005168 = (__int64)&dword_180005170;
    sub_180003740(&dword_180005278, 0, 64);
    qword_180005280 = 0;
    dword_180005288 = 0;
    dword_180005278 = 1;
    KeInitializeEvent(&Event, SynchronizationEvent, 0);
    result = CS_PcmInit();
    if ( (int)result < 0 )
    {
        _mm_lfence();
        return result;
    }
}

```

named-callback singleton: first loader owns the context, others rendezvous

```

if ( MEMORY[0xFFFFF78000000330] )
{
    *(_BYTE *) (qword_180005168 + 4424) = 0;
}
else
{
    CS_MemMove(
        (char *) (qword_180005168 + 328),
        (char *)&retaddr,
        ((unsigned int)&retaddr & 0xFFFFF000) - (unsigned int)&retaddr + 4096);
    *(_BYTE *) (qword_180005168 + 4424) = 1;
}

```

the actual ‘pinned’ 4 KB code page being copied into resident context.

2. The exported ABI (ordinals 800–806)

Ord	Renamed	Purpose
800	Pcm_AllocPool	pool alloc (NonPaged unless flag)
801	Pcm_FreePool	pool free
802	Pcm_RegisterCallbackSlot	register a callback in a 16-slot table
803	Pcm_ReadPinnedPage	copy the resident 4 KB pinned page out
804	Pcm_GetContextPtr	return context base
805	Pcm_RunCleanup	invoke CS_CleanupEntries
806	Pcm_InitApc	initialize a KAPC for injection

```

Pcm_RegisterCallbackSlot(id, fn, &old):    // ord 802  the interesting
one
    ExAcquireFastMutexUnsafe(shared+264);
    slot = 0;
    while (table[slot].id != id && table[slot].id != 0 && slot < 16)
        slot++;
    *old = table[slot].fn;                  // return previous handler
    table[slot].fn = fn;  table[slot].id = id;  // install new handler

```

A fixed **16-entry {id, fn} table** under a fast mutex. Other CS drivers register handlers here by id; the broker dispatches. Pcm_InitApc (0x180001340) wraps KeInitializeApc cspcm4 owns the **APC machinery** the sensor uses to run code in a target thread’s context.

```

void __fastcall Pcm_RegisterCallbackSlot(int a1, __int64 a2, _QWORD *a3)
{
    __int64 v4; // rbx
    unsigned int v7; // eax
    __int64 v8; // r9
    _DWORD *v9; // r8
    __int64 v10; // rcx

    *a3 = 0;
    v4 = qword_180005168;
    KeEnterCriticalRegion();
    ExAcquireFastMutexUnsafe((PFAST_MUTEX)(v4 + 264));
    *(_QWORD *)(v4 + 320) = KeGetCurrentThread();
    v7 = 0;
    v8 = qword_180005168;
    v9 = (_DWORD *) (qword_180005168 + 8);
    while ( *v9 != a1 && *v9 )
    {
        ++v7;
        v9 += 4;
        if ( v7 >= 0x10 )
            goto LABEL_7;
    }
    v10 = 2LL * v7;
    *a3 = *(_QWORD *) (qword_180005168 + 16LL * v7 + 16);
    *(_QWORD *) (qword_180005168 + 8 * v10 + 16) = a2;
    *(_DWORD *) (qword_180005168 + 8 * v10 + 8) = a1;
    v8 = qword_180005168;
LABEL_7:
    *(_QWORD *) (v8 + 320) = 0;
    ExReleaseFastMutexUnsafe((PFAST_MUTEX)(v8 + 264));
    KeLeaveCriticalRegion();
}

```

cspcm4's inter-module callback registry 16 slots

```

{
    __int64 (__fastcall *v11)(); // r10

    if ( a7 || (v11 = sub_1800012C0, !a6) )
        v11 = a6;
    KeInitializeApc(a1, a2, a3, sub_180001180, (unsigned __int64)sub_180001260 & -(__int64)(a5 != 0), v11, a7, a1);
    a1[15] = a8;
    a1[12] = a4;
    a1[13] = a5;
    a1[11] = a9;
    a1[14] = a6;
    return a9;
}

```

the sensor's APC-injection primitive lives here.

3. The 6 embedded SHA-1 strings

.rdata holds **six 40-char SHA-1 hex strings** collected into a pointer array at 0x1800044E0. Honest note: in this module the array has **no direct code xref** nothing in cspcm4's .text reads it. That implies the strings are **identity/allowlist data consumed elsewhere** (read via the pinned page by csagent, or matched by higher-level integrity logic). They look exactly like an **allowlist of trusted module SHA-1s** but the comparison isn't performed here.

.rdata:00000001800044E0	dq offset a3d292bf1afefeb ; "3d292bf1afefeb6d13a9b4be18a8f1fd73e83f8"...
.rdata:00000001800044E8	dq offset a752d6341d920ff ; "752d6341d920ffa63245767961896a69ad2dd29"...
.rdata:00000001800044F0	dq offset a427c5fa1b2c2e8 ; "427c5fa1b2c2e8c8de14e164d89046e2828f1c1"...
.rdata:00000001800044F8	dq offset aAfe3db7a5a86deb ; "afe3db7a5a86deb422ba7aa55ba4172426c60fc"...
.rdata:0000000180004500	dq offset aF5fb21761531ff ; "f5fb21761531fff914d1e8dc1436e59cd311427"...
.rdata:0000000180004508	dq offset a6425229a9b80c7 ; "6425229a9b80c7832492123fcc55217ae687dff"...

embedded SHA-1 allowlist data only, compared by the consumer, not here.

Reframe of the anti-tamper picture: cspcm4 is *infrastructure*, not an active guard. No code here watches csagent's memory or re-verifies it at runtime. Patching CS_IsSensorEnabled→1 in csagent is **not** stopped by cspcm4. The real tamper-resistance lives in the user-mode service + ELAM/early-launch, PPL on CSFalconService.exe, and whatever consumes the SHA-1 allowlist. cspcm4 is plumbing, not a wall.

Section 7 User-Mode Detection (CSFalconService.exe)

Binary: **CSFalconService.exe** (SYSTEM, ~15.5 MB)

The kernel driver is only **half** the sensor. The other half does what the kernel can't easily do: DNS analysis, browser-credential-theft detection, PowerShell/script inspection via AMSI, device posture, and **Identity Protection** (Kerberos/NTLM/LDAP). The service is a modern async C++ actor-model binary (ActivatableClass<...Actor, Actors>) the RTTI class names lay the whole architecture out.

```

39 .data:00000001... 0000005A C (16 ... DNS/StartHostOrIpResolveFromDnsResolverActor
39 .data:000000014... 00000037 C .?AV?$ActivatableClass@VDnsResolverActor@Actors@@@S02@@
39 .data:000000014... 0000001E C .?AVDnsResolverActor@Actors@@@

39 .data:000000014... 00000038 C .?AV?$BasicIntrusiveReference@VPowerShellAutomation@@@
39 .data:000000014... 00000025 C .?AVIPowerShellAutomationInterface@@
39 .data:000000014... 0000001B C .?AVPowerShellAutomation@@

39 .data:000000014... 00000023 C .?AVIdpPolicyManagerActor@Actors@@
39 .data:000000014... 0000003C C .?AV?$ActivatableClass@VIdpPolicyManagerActor@Actors@@@S02@@
39 .data:000000014... 00000076 C .?AV$SLRUCache@U_GUID@@@U_TRANSACTION_STATE_VALUE@IdpPolicyManagerActor@Actors@@@U?$hash@U_GUID@@@std@@@InMemoryCache@@

```

the service's own type metadata reveals every detection subsystem.

```

CSFalconService.exe (SYSTEM)
├─ NetworkPolling ..... DNS monitoring
├─ AMSI provider ..... PowerShell / script inspection
├─ Browser policy ..... Chrome/Edge/Firefox extension + credential monitoring
├─ ZeroTrustAssessment .... device posture scoring (data.zta)
├─ IdentityProtection ..... Kerberos/NTLM/LDAP + MFA (the CsInteg RPC server)
├─ ETW consumer ..... consumes OS ETW providers
└─ NGDP / ChannelFile ..... cloud comms: pull detection content, push telemetry

```

1. DNS monitoring NetworkPolling::DnsResolver

Observable<DnsResult> pushes results to IObserver<DnsResult>; DnsQueryBatchActor batches lookups. Correlates process/network events to **domain names** (kernel WFP sees IPs; this sees names), flags C2/known-bad domains.

```

39 .data:000000014... 00000037 C .?AV?$ActivatableClass@VDnsResolverActor@Actors@@@S02@@
39 .data:000000014... 0000001E C .?AVDnsResolverActor@Actors@@@

39 .data:000000014... 00000020 C .?AVDnsQueryBatchActor@Actors@@
39 .data:000000014... 00000039 C .?AV?$ActivatableClass@VDnsQueryBatchActor@Actors@@@S02@@

```

the DNS-monitoring subsystem (NetworkPolling::DnsResolver).

2. PowerShell / script inspection AMSI provider

Registry: SOFTWARE\Microsoft\AMSI\Providers. **CrowdStrike registers itself as an AMSI provider** its scan callback receives every script buffer *decoded*, the moment it's about to execute. Catches obfuscated PowerShell, fileless/in-memory scripts, LOLBin abuse. This is why “just base64-encode your PowerShell” doesn't evade AMSI hands over the decoded script, not the encoded command line.

```

39 .data:00000001... 00000052 C (16 ... IDP:Telemetry.ResetHistogramsIntervalMin
39 .data:00000001... 0000004E C (16 ... SOFTWARE\\%sMicrosoft\\AMSI\\Providers\\%s
39 .data:00000001... 0000005C C (16 ... SOFTWARE\\WOW6432Node\\Microsoft\\AMSI\\Providers
39 .data:00000001... 00000044 C (16 ... SOFTWARE\\Microsoft\\AMSI\\Providers

39 .data:000000014... 00000038 C .?AV?$BasicIntrusiveReference@VPowerShellAutomation@@@
39 .data:000000014... 00000025 C .?AVIPowerShellAutomationInterface@@
39 .data:000000014... 0000001B C .?AVPowerShellAutomation@@

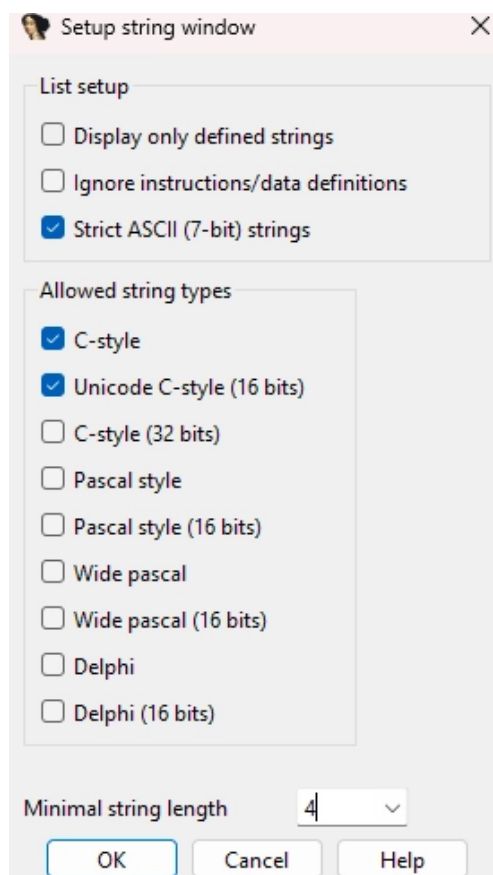
39 .data:000000014... 0000001F C .?AVIPowerShellScriptHandler@@

```

CrowdStrike registers as an AMSI provider scripts arrive decoded.

3. Browser monitoring extensions + credentials

Policy keys Software\Policies\Google\Chrome\ExtensionSettings (+ allow/block, NativeMessagingAllowlist), Edge + Firefox equivalents, plus browser **User Data** paths where Login Data/Cookies live. Extension governance + **infostealer detection** (awareness of browser credential stores). Kernel Object callback stops LSASS dumping; this watches the *browser* stores.



00000001...	00000040	C (16 ... Software\Policies\Google\Chrome
00000001...	00000072	C (16 ... Software\Policies\Google\Chrome\NativeMessagingAllowlist
00000001...	00000064	C (16 ... Software\Policies\Google\Chrome\ExtensionSettings
00000001...	000000A6	C (16 ... Software\Policies\Google\Chrome\ExtensionSettings\klicnoclcmhafeepkdpdpjijlaaff
00000001...	000000A6	C (16 ... Software\Policies\Google\Chrome\ExtensionSettings\obojibenopmhbabffmmcimgingecokj
00000001...	00000080	C (16 ... \SOFTWARE\Microsoft\Windows\CurrentVersion\App Paths\chrome.exe
00000001...	00000084	C (16 ... Software\Policies\Google\Chrome\PolicyListMultipleSourceMergeList
00000001...	00000068	C (16 ... Software\Policies\Google\Chrome\ExtensionSettings*
00000001...	00000074	C (16 ... Software\Policies\Google\Chrome\ExtensionInstallAllowlist
00000001...	00000064	C (16 ... Software\Policies\Google\Chrome\ExtensionSettings
00000001...	00000074	C (16 ... Software\Policies\Google\Chrome\ExtensionInstallBlocklist
00000001...	00000058	C (16 ... \AppData\Local\Google\Chrome Beta\User Data
00000001...	00000056	C (16 ... \AppData\Local\Google\Chrome Dev\User Data
00000001...	0000004E	C (16 ... \AppData\Local\Google\Chrome\User Data

browser extension governance + credential-store monitoring.

4. ZeroTrust Assessment

%ProgramData%\CrowdStrike\ZeroTrustAssessment\data.zta a device posture/compliance score feeding conditional access. Trust scoring, a detection input (compromised/misconfigured device scores low).

5. Identity Protection the big user-mode engine

Largest user-mode subsystem, behind the RPC server. EndpointQueryActor/EndpointServer answer endpoint-enrichment queries over the **CsIntegRpcServer** RPC interface (UUID 88F4E894-942A-40D8-BA06-89442D8DA600). IdpPolicyManagerActor + IValueProvider<...> evaluate rules against

identity/endpoint attributes; IdpMfaUiManagerActor drives the MFA prompt. Catches **identity attacks** pass-the-hash, Kerberoasting fallout, anomalous NTLM, lateral movement and can force step-up MFA.

Process	Source	Type	String
...	...	...	...
...	...	C	..?AVIdpPolicyManagerActor@Actors@@
...	...	C	..?AV?SActivatableClass@VldpPolicyManagerActor@Actors@@@S02@@
...	...	C	..?AV?SLRUCache@U_GUID@@@U_TRANSACTION_STATE_VALUE@IdpPolicyManagerActor@Actors@@@U?Shash@U_GUID@@@std@@@InMemoryCache@@

Falcon Identity Threat Detection the endpoint-side actors.

1	const CsIntegration::EndpointServer::vftable'	0000000140C1D0F8
1	CsIntegration::EndpointServer::RTTI Class Hierarchy Descriptor'	0000000140CCA398
1	const CsIntegration::EndpointServer::RTTI Complete Object Locat...	0000000140CCA3B0
1	CsIntegration::EndpointServer::RTTI Base Class Descriptor at (0,-1,...	0000000140CCA3D8
1	CsIntegration::EndpointServer::RTTI Base Class Array'	0000000140CCA400
1	CsIntegration::EndpointServer::RTTI Type Descriptor'	0000000140DF9FE8

the CsIntegration EndpointServer the RPC handler implementation.

6. ETW consumption + cloud content

EtwConsumer@ETW (OpenTraceW, ProcessTrace) consumes OS ETW providers to enrich telemetry.

NGDP / ChannelFile (ODSChannelFileActor, ChannelFileUpdated) downloads **channel files** detection content/policy/IOC updates via WinHTTP and applies them at runtime. Same channel-file mechanism famous from July 2024. Cloud transport = WinHTTP.

7. The IPC architecture (and the trust model)

- **Named pipes:** \\.\pipe\CrowdStrike\, \\.\pipe\CS\.
- **RPC:** CsIntegRpcServer on ncalrpc, RPC_IF_ALLOW_SECURE_ONLY, RpcServerRegisterAuthInfoW (NTLM/Negotiate), + a security callback requiring PKT_PRIVACY.

```

if ( a3 )
    v4 = RpcServerUseProtseqEpW((RPC_WSTR)L"ncalrpc", 0xAu, a3, 0);
else
    v4 = RpcServerUseProtseqW((RPC_WSTR)L"ncalrpc", 0xAu, 0);

{
    IfCallback = Cs_RpcSecurityCallback;
    v11 = (void *)*((_QWORD *)ServerPrincName + 4);
    if ( *((_QWORD *)ServerPrincName + 7) )
        IfCallback = (__int64 (__fastcall *) (__int64, void *)) *((_QWORD *)ServerPrincName + 7);
    v12 = RpcServerRegisterIfEx(v11, 0, 0, 8u, 0x4D2u, (RPC_IF_CALLBACK_FN *)IfCallback);
}

```

the CsInteg RPC interface registration.

```

if ( RpcBindingInqAuthClientW(a2, 0, 0, &AuthnLevel, &AuthnSvc, 0) || AuthnLevel != 6 )
    return 5;
result = 5;
if ( AuthnSvc == 10 )
    return 0;
return result;
}

```

the per-call gate checks transport auth level, **not caller identity** (authentication without authorization

The security model is “**authenticated + encrypted transport**” it verifies the channel is secure and authenticates the caller, but in this callback does not restrict *which* users may call.

Section 8 The Decision Map (how a verdict routes)

Binary: **CSAgent.sys**

We've seen every place the sensor *sees* you. Now: when an event is built, how does it become a kill/allow decision?

csagent.sys · 0x140377450

CS_IsSensorEnabled()	body runs when zero
g_DetectionEngine	must be non null
g_SensorConfig + 56	rules loaded flag
CS_AcquireEventBuffer	buffer type 1002

CS_BuildCreationEvent	552 byte pool allocation
-----------------------	--------------------------

via CS_EventGetField · 0x140378F80

72	engine call, produces verdict
6	image hash present
10	signature present
5186 (0x1442)	commit, requires 6 and 10

if (flags & 2) *a4 = verdict	only write to out param
CS_DetectInjection	after ValidateCreatorThread

The event is a C++-style object

The one vtable that matters is at event+32. Everything downstream talks through two calls: CS_EventGetId (event sequence ID) and CS_EventGetField (0x140378F80) a big switch(selector) returning/lazily-computing a field. The 72 / 6 / 10 / 5186 numbers are **method selectors** on this object.

```

switch ( (unsigned __int16)a2 )
{
    case 0xAu:
        return a1 + 34;
    case 1u:
        return a1 + 54;
    case 4u:
        return (_QWORD *)(a1[27] + 64LL);
    case 5u:
        return (_QWORD *)(a1[27] + 8LL);
    case 6u:
        return a1 + 33;
    case 8u:
        return a1 + 35;
    case 9u:
        return a1 + 36;
}

```

the object's method table decoded.

Where detection ACTUALLY fires selector 72

```

case 72:
    if (!*((BYTE*)a1 + 310)) {                // not computed yet
        CS_EventGetField(a1, 10);             // force signature resolution
        CS_EventGetField(a1, 6);             // force hash resolution
        v4 = a1[21];                         // the RULE ENGINE context object
        v7 = *((v4) + 16);                   // engine vtable slot +16 = THE MATCHER
        v10 = v7(v4, 72, id, hash, sig, a1+37); // ← THE DETECTION CALL. verdict →
    }
    *((BYTE*)a1 + 310) = 1;
}

```

That indirect call is the actual **match-against-cloud-rules** call. The matcher is inside the loaded content object, not in csagent. Keyed on **file hash + signature + event context**. 0xC0000065 (“no matching rule”) = clean, not logged.

```

if ( *((_BYTE *)a1 + 310) )
{
    v9 = a1 + 37;
}
else
{
    (*(void (__fastcall *)(_QWORD *, __int64, _QWORD)))(*a1 + 8LL))(a1, 10, (unsigned int)(unsigned __int16)a2 - 72);
    (*(void (__fastcall *)(_QWORD *, __int64)))(*a1 + 8LL))(a1, 6);
    v4 = a1[21];
    v5 = a1[33];
    v6 = a1[34];
    v7 = *((__int64 (__fastcall *))(__int64, __int64, _QWORD, __int64, __int64, _QWORD))(*(_QWORD *)v4 + 16LL);
    v8 = *((__int64 (__fastcall *)(_QWORD *))a1)(a1);
    v9 = a1 + 37;
    v10 = v7(v4, 72, v8, v6, v5, a1 + 37);
    if ( v10 < 0 )
    {
        if ( v10 != -1073741275
            && Process != (PEPROCESS)&Process
            && *((_DWORD *)Process + 11) & 2 != 0
            && *((_BYTE *)Process + 41) >= 2u )
        {
            _mm_lfence();
            CS_DBGLogVal((__QWORD *)Process + 3, 0xFu, (__int64)&unk_1402C54C8, v10);
        }
        *v9 = 0;
    }
    *((_BYTE *)a1 + 310) = 1;
}

```

where ‘is this malware?’ gets answered.

The hash/sig gate and the bypass it creates

Full rule matching (5186) only runs when **both** the image hash and the Authenticode signature resolved. If either fails, the sensor logs error 14 and *skips* the deep match. Anything that makes hash-or-signature resolution fail (unreadable/locked image, resolution race during spin-up) drops the process into the “incomplete data” path with a weaker verdict.

The kill write-back

```
if ( (BYTE12(v42) & 2) != 0 ) // "block" flag set by the engine
    *v39 = DWORD2(v41);      // write verdict NTSTATUS to caller's out-
param
```

v39 is the same value copied into **CreateInfo->CreationStatus (+64)**. Negative NTSTATUS there = Windows aborts process creation. Block chain: **engine sets bit 1 → verdict copied → written to CreationStatus → process never runs.**

```

LABEL_66:
v35 = (*(__int64 (__fastcall **)(_QWORD *))v9)(v9);
LOBYTE(v10) = (*(__int64 (__fastcall **)(_QWORD *, _QWORD *))(*v40 + 8LL))(v40, v9);
if ( (BYTE12(v42) & 2) != 0 )
    *v39 = DWORD2(v41);

    v41 = 0;
    LODWORD(v41) = 2097872;
    v42 = 0;
    DWORD2(v42) = 16778217;
```

verdict ⇒ block chain

The injection / remote-thread tail

If not blocked, dispatch checks *who created it*:

```
CS_ValidateCreatorThread(FileObject, &flag); // control code 0x86968
if (!flag)
    CS_DetectInjection(FileObject, PsGetCurrentProcessId(), PsGetCurrentThreadId(), ...); //
0x8696C
```

Code	Function	Meaning
0x86968	CS_ValidateCreatorThread	is the creating thread a legit parent, or a hijacked/remote thread?
0x8696C	CS_DetectInjection	full cross-process injection check

How CrowdStrike catches **process hollowing / CreateRemoteThread launches** the callback fires in the injector's context, and these codes ask the filter "does the creator match the image?"

```

v20 = CS_ValidateCreatorThread(v19, v34);
if ( v20 >= 0 )
{
    LOBYTE(v10) = v34[0];
}
else
{
    if ( Process != (PEPROCESS)&Process
        && *((_DWORD *)Process + 11) & 0x80000000 != 0
        && *((_BYTE *)Process + 41) >= 2u )
    {
        _mm_lfence();
        CS_DbgLogVal(*((_QWORD *)Process + 3), 15, &unk_1402C5448, (unsigned int)v20);
    }
    LOBYTE(v10) = 0;
}
if ( !(_BYTE)v10 )
{
    v28 = *((_QWORD *)v7 + 64);
    CurrentThreadId = PsGetCurrentThreadId();
    CurrentProcessId = PsGetCurrentProcessId();
    LOBYTE(v10) = CS_DetectInjection(*((_QWORD *)a3 + 40), CurrentProcessId, CurrentThreadId, v28);
}
```

the injection-detection layer.

Section 9 The Brain (how the verdict is actually made)

Binary: CSAgent.sys

Section 8 traced the event to a single call selector 72 on the event object and stopped. This section opens that door. It's the most architecturally important thing in the whole teardown, and it explains the July-2024 incident.

The short version

The detection rules are not in the driver. `csagent.sys` is a generic **event-collection and dispatch framework**. The thing that decides a verdict the “detection engine” is a separate object **loaded into the driver at runtime** from CrowdStrike **content** (channel files from the cloud). The driver holds it in one global pointer, `g_DetectionEngine`, and calls through a vtable. Update the content, and detection behavior changes with **no change to the sensor binary**.

```
cloud content (channel files) → CS_SetDetectionEngine(cmd 1021/2000, engineObject)
                                |
                                v
                                g_DetectionEngine ————— (vtable call)
event happens → CS_BuildCreationEvent binds engine → a1[21] |
CS_EventGetField(event, 72) → engine->vtable[+16](engine, 72, hash, sig, &verdict)
                                ▼
                                verdict → block / kill / allow / telemetry
```

1. The engine is registered at runtime, not compiled in

Walk: `CS_SetDetectionEngine (0x140378360)`.

```
CS_SetDetectionEngine(a1, cmd, subcmd, engineObject):
    if (cmd != 1021 || subcmd != 2000) return STATUS_INVALID_PARAMETER;    // command
gate
    if (!g_DetectionEngine) {
        g_DetectionEngine = engineObject;    // ← install the engine
        init = engineObject + 8 + *(int*)(*(engineObject+8)+4);
        (**init)(init);    // call its initializer
        return 0;
    }
```

The engine arrives as a fully-formed **object** via the magic command pair **(1021, 2000)**. The driver stores the pointer and calls the initializer. `CS_ResetDetectionEngine (0x140378104)` does the Reverse releases and nulls `g_DetectionEngine` so the engine can be **swapped at runtime**. Content-update mechanism in three functions.

```

{
    unsigned int v4; // ebx
    __int64 v5; // rax
    void (__fastcall **v6)(_QWORD); // rcx
    if ( a2 != 1021 || a3 != 2000 )
    {
        if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 1) != 0 && *((_BYTE *)Process + 41) >= 2u )
            sub_1400088F8(*((_QWORD *)Process + 3), 31, &unk_1402C5448, a2, a3);
        return (unsigned int)-1073741811;
    }
    v4 = 0;
    if ( g_DetectionEngine != a4 )
    {
        if ( !g_DetectionEngine )
        {
            v5 = *(_QWORD *)(a4 + 8);
            g_DetectionEngine = a4;
            v6 = (void (__fastcall **)(_QWORD))(a4 + 8 + *(int *)(v5 + 4));
            (**v6)(v6);
            return v4;
        }
        if ( Process != (PEPROCESS)&Process && *((_DWORD *)Process + 11) & 1) != 0 && *((_BYTE *)Process + 41) >= 2u )
            CS_DbgLog(*((_QWORD *)Process + 3), 0x1Eu, (__int64)&unk_1402C5448);
        return (unsigned int)-1073741811;
    }
}

```

the detection engine is *installed at runtime*, not compiled in.

2. How the verdict call fires

```

case 72: // in CS_EventGetField (0x140378F80)
    CS_EventGetField(event, 10); // force signature
    CS_EventGetField(event, 6); // force hash
    v4 = event->engineCtx; // a1[21] engine, bound at build
time
    match = *(v4 + 16); // engine vtable slot +16 = THE
MATCHER
    verdict = match(v4, 72, id, hash, sig, &event->result); // * the detection
call

```

- The matcher is `engine_vtable[+16]` a function *inside the loaded content object*, not in `csagent`. The driver has no idea how matching works; it calls slot 16 and reads a verdict.
- Inputs are **file hash + signature + event context**.

```

if ( (unsigned __int16)a2 != 72 )
{
    if ( (unsigned __int16)a2 == 133 )
        return a1 + 58;
    return (_QWORD *)sub_14021E0A0();
}
if ( *((_BYTE *)a1 + 310) )
{
    v9 = a1 + 37;
}
else
{
    (*(void (__fastcall **)(_QWORD *, __int64, _QWORD))(*a1 + 8LL))(a1, 10, (unsigned int)(unsigned __int16)a2 - 72);
    (*(void (__fastcall **)(_QWORD *, __int64))(*a1 + 8LL))(a1, 6);
    v4 = a1[21];
    v5 = a1[33];
    v6 = a1[34];
    v7 = *(__int64 (__fastcall **)(__int64, __int64, _QWORD, __int64, __int64, _QWORD))(*(_QWORD *)v4 + 16LL);
    v8 = (*(__int64 (__fastcall **)(_QWORD *))a1)(a1);
    v9 = a1 + 37;
    v10 = v7(v4, 72, v8, v6, v5, a1 + 37);
    if ( v10 < 0 )
    {
        if ( v10 != 0xC0000225
            && Process != (PEPROCESS)&Process
            && *((_DWORD *)Process + 11) & 2) != 0
            && *((_BYTE *)Process + 41) >= 2u )
        {
            _mm_lfence();
            CS_DbgLogVal(*((_QWORD *)Process + 3), 0xFu, (__int64)&unk_1402C54C8, v10);
        }
        v9 = 0;
    }
}

```

the actual malware decision a vtable call into cloud-loaded content.

3. Why this architecture matters

- **Cloud-updatable detection.** New IOCs/rules/signatures ship as **channel files** → a new engine object pushed via `CS_SetDetectionEngine`. Detection improves with **no sensor upgrade**.

- **The sensor binary is nearly rule-free.** Reversing csagent tells you *how events are collected and how the engine is called* but **not what triggers a detection**, because those rules live in encrypted cloud content. A deliberate limit on what static analysis can reveal.
- **This is the July-2024 crash surface.** The famous incident was a **content (channel file) update** the engine mis-processed, faulting inside this exact path a bad rule object pushed into g_DetectionEngine, not a code bug in the sensor. The architecture that makes detection updatable is the same one that made a bad content push global.

Mental model: csagent.sys = eyes + hands (see events, apply verdicts); the loaded content engine = the brain (decide). The brain is swappable, cloud-delivered, and absent from the binary.

Section 10 Blind Spots & Evasion Seams

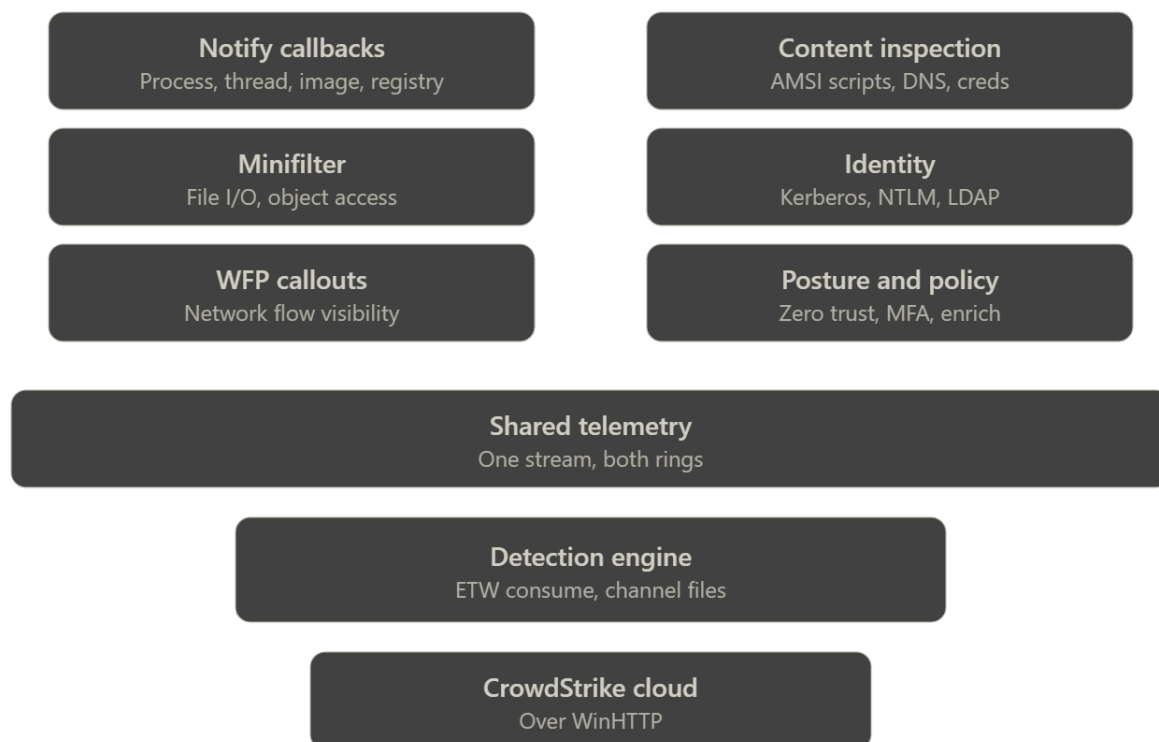
Synthesis structural observations from the code, not tested exploits. Reuse earlier screenshots (O1/O2, W1, E1, R1).

Every seam the teardown surfaced, collected in one place:

#	Seam	Where	Consequence
1	ExGetPreviousMode() trust boundary	Object (O1/O2) + Registry (R1) callbacks	Object/Registry filters only fully act on user-mode callers. A kernel-mode actor (another driver, or an in-kernel exploit) opening a handle or touching the registry is largely unfiltered. The single biggest structural assumption in the driver.
2	NULL engine / config-not-ready	CS_DetectionDispatch gates 2–3, CS_SetDetectionEngine (E1)	If g_DetectionEngine == NULL (early boot before content loads) or * (g_SensorConfig+56)==0 (rules still loading), dispatch passes everything unmatched. CS_ResetDetectionEngine nulls it. Delay/prevent engine registration = a window where the world's best rules aren't consulted.
3	Hash-or-sig resolution failure	selector-72 gate (D2/E2)	Full rule match (5186) skipped when hash or signature fails to resolve the most practical software-side gap (locked/unreadable image, spin-up race).

#	Seam	Where	Consequence
4	IRQL / APC / registration windows	thread (T1), image (I1-3), file (F1)	Thread callback bails at raised IRQL / APCs-disabled. Minifilter has an explicit “registering” state (2). Early-boot / just-mounted-volume I/O is unfiltered.
5	Config-gated detectors	byte_1402FB824 (remote-thread), g_TraceEnabled, CS_IsSensorEnabled()→0	Before policy lands or if a flag is clear, specific detections are silently off.
6	10,000-flow cap	CS_FlowEstablishClassify (W1)	Past 0x2710 tracked flows, new connections aren’t tracked (untracked = not reset-killable). Reachability is a dynamic question, but it’s a hard ceiling.
7	Reset-based kill is racy	CS_KillFlaggedFlows (W2/W3)	Termination is a TCP RST injected <i>after</i> a verdict. Short-lived flows (a quick beacon) may complete before the reset lands. Mitigation after detection, not prevention.
8	DPI sees a copy	FwpsCloneStreamData0	Classic DPI evasions (segmentation, out-of-order, TLS) limit payload matching. Encrypted C2 is opaque only metadata/JA3-style signals remain.
9	Self / exclusion allowlists	CS_IsExcludedProcess, Ob self-access, registry recursion guard	“Trusted” contexts. Enumerating what’s trusted = finding the gaps.
10	Static fingerprints	WFP callout GUIDs, FLT altitude, Ob/Cm altitudes, pool tags (Tes1/Tes2, PdPp, CSFR)	All enumerable (netsh wfp show state, fltmc filters, !poolused) confirm the sensor and its coverage before acting.
11	Content-driven = binary can’t reveal rules	g_DetectionEngine (Section 9)	Evasion by “reading the binary for the rule” doesn’t work the rule isn’t there. You’d need the decrypted channel content. The <i>framework gates</i> are in the binary; the <i>rules</i> aren’t.
12	In-kernel parsers on attacker data	PE headers (I2), stream payloads, file-name info	Parse/allocation bugs here would be high-value the memory-safety attack surface.

So end to end



Appendix Function Rename Map

CSAgent.sys

Address	Name	Role
0x140377450	CS_DetectionDispatch	the decision function
0x14037889C	CS_BuildCreationEvent	552-byte creation event builder
0x140379454	CS_BuildTerminationEvent	344-byte termination event builder
0x140378F80	CS_EventGetField	event object selector dispatcher
0x14001FAB0	CS_EventGetId	returns event sequence id
0x140378360	CS_SetDetectionEngine	installs the runtime engine (cmd 1021/2000)
0x140378104	CS_ResetDetectionEngine	tears down / swaps the engine
0x140378704	CS_DetectInjection	cross-process injection check (0x8696C)
0x140099DF0	CS_ValidateCreatorThread	creator-thread legitimacy (0x86968)
0x1400C4784	CS_RefProcessRecord	refcount++ on record
0x1400C4DA0	CS_ResolveProcessImageInfo	lazy image-path field

Address	Name	Role
0x1400C4DF8	CS_ResolveProcessCmdInfo	lazy command-line field
0x1400C4F78	CS_GetProcessTokenInfo	SID / integrity resolver
0x1400CFE50	CS_ThreadCreateCallback	thread create/exit callback
0x1400D190C	CS_DetectRemoteThread	remote-thread check
0x1400B03A0	CS_LoadImageCallback	image/DLL/driver load
0x1400B1CB8	CS_HandleDriverLoad	driver-load path (BYOVD)
0x1400C20D0	CS_0bPreOpCallback	handle pre-op filter
0x1400C2234	CS_CheckHandleAccess	access-mask decision (LSASS shield)
0x1400C7790	CS_RegistryCallback	registry filter
0x14020DE50	CS_RegisterCallouts_Eng1	WFP callouts → engine 1
0x14021E738	CS_RegisterCallouts_Eng2	WFP callouts → engine 2
0x1400B5DF0	CS_FlowEstablishClassify	allocates CSFR flow record
0x1401D108C	CS_KillFlaggedFlows	enumerates flagged flows
0x1401D626C	CS_ResetFlow	injects a TCP reset
0x1401D4260	CS_InjectTransportPacket	transport/stream injection (C2 kill)
0x1403630B0	CS_RegisterMinifilter	FltRegisterFilter + arm
0x1403622A0	CS_FltPreCreate	pre IRP_MJ_CREATE
0x14009E2F4	CS_HandleFileSetInfo	delete/rename handler
0x1400A2834	CS_HandleFileCreate	CREATE handler

Globals: g_DetectionEngine 0x140515060 · g_SensorConfig 0x140515030 · g_TraceEnabled 0x1402FA081 · g_WfpEngine1 0x1402FB328 · g_WfpEngine2 0x1402FB370 · g_CalloutTable 0x1402F9A48

cspcm4.sys

Address	Name	Role
0x180001470	DllInitialize	rendezvous / init
0x180001010	Pcm_BrokerCallback	hands out shared context
0x1800010A0	Pcm_RegisterCallbackSlot (ord 802)	16-slot callback registry

Address	Name	Role
0x180001400	Pcm_ReadPinnedPage (ord 803)	read pinned 4 KB page
0x180001340	Pcm_InitApc (ord 806)	APC injection init
0x180003480	CS_MemMove	SSE memmove (page copy)

CSFalconService.exe

Address	Name	Role
0x1405B2FF0	Cs_RpcServerStart	registers the CsInteg RPC interface
0x1405B3380	Cs_RpcSecurityCallback	per-call RPC security check

X : 0xDbgman